The present work was submitted to the

Visual Computing Institute
Faculty of Mathematics, Computer Science and Natural Sciences
RWTH Aachen University

Analysis and Comparison of Transfer Functions in Hybrid Eulerian-Lagrangian Simulation Methods

Master Thesis

presented by

Torben Steegmann
Student ID Number 415378

May 2026

First Examiner: Prof. Dr. Jan Bender
Second Examiner: Prof. Dr. Torsten Wolfgang Kuhlen

Hiermit versichere ich, diese Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt, sowie Zitate kenntlich gemacht zu haben.

I hereby affirm that I composed this work independently and used no other than the specified sources and tools and that I marked all quotes as such.

Aachen, June 29, 2026

Torben Steegmann

Abstract

Hybrid particle-grid methods are widely used to simulate fluids and related materials in computer graphics, where motion must remain stable over time while preserving fine detail. These methods repeatedly transfer velocity between moving particles and a background grid, and the rule governing that transfer largely decides the trade-off between stability and detail. The Particle-in-Cell (PIC) method is stable but smooths motion away, while the Fluid-Implicit-Particle (FLIP) method preserves detail but suffers from ringing, a persistent low-speed oscillation. The Affine Particle-in-Cell (APIC) method was introduced to reconcile the two, and the Polynomial Particle-in-Cell (PolyPIC) method extends it further.

This thesis compares these four transfer methods within a single custom solver in which only the transfer step varies, so that every measured difference is attributable to the transfer rule rather than to the surrounding setup. The methods are evaluated across a catalog of controlled scenes using recorded diagnostics, such as energy dissipation, angular momentum, and a measure of ringing, rather than rendered appearance alone.

APIC retains PIC's stability, approaches FLIP's detail, and conserves angular momentum better than both, without necessitating per-scene tuning. PIC's stability is found to be over-damping rather than faithful preservation, and FLIP's detail comes with persistent ringing. PolyPIC's gain over APIC is narrow and emerges only where higher-order velocity structure is strong. The added per-particle state, rather than transfer runtime, is the main cost of the richer methods.

Contents

Chapter 1

Introduction

Fluids shape many of the systems people want to predict, design, and depict: storms, ocean waves, aircraft wakes, cooling flows, smoke, fire, splashes, and breaking water all depend on motion that changes continuously over time. This makes fluid simulation valuable in science and engineering, where direct experiments may be expensive, dangerous, or difficult to repeat. It is also an important topic for computer graphics, where motion of fluid and materials must be believable to the viewer. The same complexity that makes fluids visually and physically rich also makes them difficult to compute. Practical methods therefore have to choose how fluid motion is represented, how information is transported, and how much numerical error can be tolerated before the simulated motion loses detail or becomes unstable. This thesis studies that problem through hybrid particle-grid methods, with a focus on transfer quality and the simulation-quality criteria used to compare them.

1.1 Motivation

Fluids are central to many natural and engineered systems because they transport momentum, heat, mass, and visible material through space. Their motion matters in weather and climate modeling, ocean and coastal dynamics, aerodynamic and hydrodynamic design, cooling systems, industrial processes, smoke transport, and fire-driven flows [Wen08; KDK+19]. In these settings, simulation makes it possible to examine fluid behavior under controlled assumptions, especially when physical tests would provide only limited measurements or would be difficult to repeat under comparable conditions.

In computer graphics, fluid simulation has a different but related role. Visual effects, animation, and interactive media use simulated fluids for phenomena such as water, smoke, fire, foam, splashes, pouring liquids, and breaking waves [Bri15].

As these effects are not judged only by isolated still images, the motion must remain coherent over time, respond plausibly to scene geometry and external forces, and preserve enough detail to remain convincing in motion.

This makes simulation an interesting topic for this thesis. While purely hand-authored or ad hoc visual effects can be useful in narrow production settings, they do not provide a clean basis for comparing numerical behavior across methods. A reusable simulation model can instead be run under controlled scene settings, which makes it possible to reason about stability, detail preservation, artifacts, runtime, and memory use.

Fluid simulation remains challenging because these criteria often conflict. A stable method may remove too much motion detail, while a method that preserves detail may introduce noise or unstable oscillations. Different applications also prioritize different compromises. Engineering simulations may emphasize accuracy and validation, visual effects may emphasize controllability and appearance, and interactive applications may emphasize speed and robustness. This thesis starts from this tension and evaluates selected fluid simulation methods by how their behavior develops over time, not only by whether a single frame appears plausible.

1.2 Thesis Scope

This thesis focuses on hybrid methods for fluid simulation. These methods combine moving particles with a grid-based representation. The scope is therefore not a general survey of all fluid solvers. Instead, the thesis narrows from fluid simulation to hybrid particle-grid simulation and then to selected transfer variants.

The comparison centers on PIC, FLIP, APIC, and PolyPIC. PIC provides the historical baseline, FLIP is an important variant for reducing dissipation, and APIC is the central transfer method investigated in this thesis [Har62; BR86; JSS+15]. PolyPIC extends APIC by replacing its affine particle information with a higher-order polynomial representation, and is included to test whether APIC is the endpoint of this line of work or one step in it [FGG+17]. APIC is also relevant beyond the original method paper. Disney Animation's water work for *Moana* provides a production example of APIC-based fluid simulation in feature animation [FSN17]. Later production discussion notes that APIC became part of Houdini's stock toolset and was built upon in the *Moana 2* water workflow [Li24].

The primary application domain is fluid simulation, but related hybrid methods for materials such as sand, deformable solids, and other continuum-like media are also studied, because they can be simulated in a very similar way [ZB05; SCS94; SSC+13]. The thesis explains the relevant theory, but theory is not the only contribution. We also use a custom C++ physics engine to produce controlled examples and visual comparisons. The implementation is meant to support the

method comparison experimentally and visually, not merely to demonstrate that the methods can be programmed.

The implementation gives control over which internal simulation data can be inspected during evaluation. Instead of relying only on rendered output, the thesis can compare methods through controlled scene settings, visible artifacts, stability over time, detail preservation, and recorded diagnostic quantities. This makes the implementation a tool for analysis, as it exposes the state needed to study how the transfer methods behave, where they lose information, and when unwanted artifacts appear.

1.3 Transfer Quality As An Evaluation Problem

Hybrid particle-grid methods use two representations for one simulated motion. Moving samples carry material information through the flow, while a grid or spatial field provides a structured representation for operations that are more naturally organized in space. The important consequence for this thesis is that information must be exchanged between these representations. This particle-grid transfer step is not only bookkeeping because it fundamentally determines which information survives as the simulation advances.

This makes transfer behavior a central object of the thesis comparison. A transfer method can preserve, smooth, distort, or destabilize velocity information, rotational motion, and small-scale motion detail. The selected methods make different compromises. PIC provides a dissipative baseline, FLIP preserves more motion detail but can retain more noise, and APIC is the central method in this thesis because it carries additional local motion information [Har62; BR86; JSS+15]. PolyPIC carries this idea further, enriching each particle with higher-order polynomial information so that more of the motion the grid can represent survives the transfer [FGG+17]. The exact transfer operators, conservation properties, and affine state are introduced later. Here, the focus is on the visible and measurable behavior that follows from each transfer choice.

The comparison is guided by stability, preservation of motion detail, numerical dissipation, noise, visible artifacts, and reproducibility under comparable scene settings. Stability asks whether errors remain bounded or grow into visible artifacts as the simulation evolves. Detail preservation asks whether motion and small-scale structures survive the repeated exchange between particles and grid. Ringing instability is the primary artifact and quality criterion in this thesis. At this stage, it is treated as unwanted oscillatory behavior that affects the simulated motion over time.

Runtime and memory use are secondary practical criteria. They matter because transfer methods may trade simulation behavior against additional computation or

stored state, but the detailed costs belong with the implementation and evaluation. For the Introduction, the central point is that transfer quality shapes the visible and measurable behavior of the simulation over time.

1.4 Structure Of The Thesis

The remainder of this thesis is structured as follows. Chapter 2 places the investigated transfer methods within the development of Eulerian, Lagrangian, and hybrid particle-grid simulation. Chapter 3 introduces the mathematical model and the shared particle-grid solver cycle needed for the comparison. Chapter 4 then defines the PIC, FLIP, APIC, and PolyPIC transfer rules, establishes their relevant properties, and presents the criteria and metrics used to compare them. Chapter 5 evaluates these methods in controlled fluid and deformable-material experiments, with particular attention to stability, ringing, numerical dissipation, detail preservation, and computational cost in two and three dimensions. Finally, Chapter 6 summarizes the findings, discusses the limitations of the investigation, and outlines directions for future work.

Chapter 2

Related Work

Particle-grid simulation methods have a long history in computational fluid dynamics and computer graphics. The particle-in-cell idea appears as early as the 1960s, including the 1962 technical report *The Particle-in-Cell Method for Numerical Solution of Problems in Fluid Dynamics* [Har62]. This demonstrates that the transfer problems studied in this thesis are not a recent implementation detail, but part of a research line that has remained active for more than six decades. The central question of this thesis is not whether particles and grids can be combined at all, but how the exchange of information between them affects stability, dissipation, detail preservation, and visible artifacts.

The methods considered in this thesis sit inside a broader landscape of Eulerian, Lagrangian, and hybrid approaches. Eulerian methods store quantities on spatial fields and remain central for grid-based fluid solvers, with the marker-and-cell method providing an important early reference point for incompressible free-surface flow and Bridson's book serving as a practical graphics-oriented treatment [HW+65; Bri15]. Lagrangian methods follow moving material samples and are attractive when material motion and advection are important. Hybrid particle-grid methods combine these viewpoints and therefore inherit both their advantages but introduce transfer-related difficulties. Within this hybrid family, PIC provides an early baseline, FLIP reduces some of PIC's numerical dissipation, and APIC introduces affine particle information to improve transfer behavior [BR86; ZB05; JSS+15]. PolyPIC later extends this line of work by replacing APIC's affine model with a higher-order polynomial representation, which shows that APIC is not the endpoint of particle-grid transfer research [FGG+17]. Material point methods extend the same particle-grid idea beyond fluid velocity transfer by carrying material history on particles. Because the same particle-grid transfer studied in this thesis governs that exchange too, they remain directly relevant even though the evaluation centres on fluid simulation [SCS94; SSC+13].

This chapter gives a compact overview of the related method families before focusing on the particle-grid methods most relevant to the evaluation. It first situates Eulerian, Lagrangian, and hybrid simulation approaches, then narrows to PIC, FLIP, APIC, PolyPIC, and material-point methods. Detailed definitions, equations, and implementation choices are deferred to the Background and Method chapters, where they can be introduced in the order needed for the actual comparison.

2.1 Particle And Lagrangian Methods

Lagrangian particle methods describe fluid motion through material samples that move with the flow. Smoothed particle hydrodynamics (SPH) is a prominent example of this viewpoint and is commonly introduced as a mesh-free Lagrangian method [Mon05]. In graphics and simulation practice, SPH has been applied to interactive fluid animation [MCG03]. It also appears in biomedical blood-flow and virtual-surgery scenarios [MST04; CML+17] and in coastal-engineering free-surface problems [BCDG13]. These examples show that Lagrangian methods are not tied to a single accuracy or runtime regime and instead are used whenever moving material samples are a useful representation. At the same time, purely particle-based methods do not provide the same regular spatial structure as grid-based methods, which makes some global field operations less direct.

Müller et al. 2003 Müller et al. introduced an SPH-based fluid simulation method for interactive applications in computer graphics [MCG03]. The paper is useful in this context because it shows the practical appeal of Lagrangian particle methods for visual free-surface motion and interactive settings. Rather than serving as a direct baseline for this thesis, it establishes SPH as a representative particle-based alternative to grid-centered fluid simulation.

Bender and Koschier Bender and Koschier later proposed divergence-free SPH for incompressible and viscous fluids [BK16]. Compared with earlier interactive SPH work, this method focuses more directly on incompressibility and on reducing divergence in the velocity field. It therefore illustrates how later Lagrangian methods address physical constraints and solver robustness more explicitly. This contrast leads naturally to Eulerian grid-based solvers, where incompressibility and spatial field operations are organized on a fixed discretization.

2.2 Eulerian Grid-Based Solvers

Eulerian fluid solvers represent quantities such as velocity and pressure as fields over fixed spatial locations. In this viewpoint, the fluid moves through the spatial discretization rather than carrying the whole representation on moving samples. This regular structure is useful for field operations, boundary handling, and incompressibility-related computations. Eulerian and grid-based methods remain relevant in graphics, where Bridson presents them as a central practical framework for fluid simulation [Bri15], and in atmospheric modeling, where Kühnlein et al. describe a finite-volume dynamical core for numerical weather prediction [KDK+19]. Across these settings, the common feature is not a single accuracy or runtime target, but the usefulness of a spatial field representation for the computation being performed.

Harlow and Welch 1965 Harlow and Welch introduced the marker-and-cell method for numerical calculation of time-dependent viscous incompressible flow with a free surface [HW+65]. The paper is an important early grid-based reference point because it organizes the fluid computation around a spatial discretization rather than around particles as the primary representation. For this Related Work section, the method is relevant mainly as historical context for Eulerian incompressible free-surface solvers. The detailed staggered-grid layout and pressure solve are deferred to the Background chapter.

Bridson 2015 Bridson's book gives a practical graphics-oriented treatment of grid-based fluid simulation [Bri15]. Although the presentation is primarily Eulerian, it remains useful for this thesis because many grid-side operations also appear in hybrid particle-grid solvers. In particular, the grid used by hybrid methods provides the setting for operations such as interpolation, boundary handling, and incompressibility enforcement, while particles carry advected material information. Bridson is therefore used here as background for the Eulerian side of the method landscape, whereas transfer-specific claims are left to the hybrid particle-grid literature discussed next.

2.3 Hybrid Particle-Grid Methods

Hybrid particle-grid methods combine the two viewpoints introduced in the previous sections. Particles move with the simulated material and can carry advected quantities, while the grid provides a structured domain for spatial operations such as force computation, interpolation, and incompressibility-related constraints. This combination is useful in fluid simulation and in granular or sand-like graphics

simulation, where moving material detail and grid-based computation both play important roles [Har62; ZB05; JSS+15]. The same broad particle-grid idea also appears in material point methods, which are discussed separately in Section 2.4. For this thesis, the central issue inside the hybrid family is transfer quality: each method makes a different choice about what information is preserved, smoothed, or destabilized when data moves between particles and grid.

Harlow 1962 Harlow's particle-in-cell method is an early particle-grid method and forms the historical baseline for the methods considered in this thesis [Har62]. In the present context, PIC is relevant because it establishes the basic idea of combining particles with cells. Later work often treats PIC-style transfer as the dissipative baseline that FLIP and APIC modify in order to preserve more information across the transfer step [BR86; JSS+15].

Brackbill and Ruppel 1986; Zhu and Bridson 2005 Brackbill and Ruppel introduced FLIP as a particle-in-cell variant designed to reduce the dissipation associated with PIC [BR86]. Instead of treating the grid value as a complete replacement of the particle state, FLIP transfers changes from the grid back to the particles. This makes FLIP attractive for preserving motion detail, but it can also allow particle-level noise or artifacts to persist. Jiang et al. use this FLIP behavior as part of the motivation for APIC [JSS+15]. Zhu and Bridson later brought FLIP-style particle-grid transfer into computer graphics for sand animation, showing the relevance of the approach for visually rich granular and fluid-like material behavior [ZB05].

Jiang et al. 2015 Jiang et al. introduced APIC as an affine extension of particle-in-cell transfer [JSS+15]. The method augments particles with local affine velocity information. At a high level, this helps preserve local velocity variation across transfer. This makes APIC the most important method in the related-work lineage for this thesis. The detailed affine representation and transfer equations are deferred to the Method chapter, where APIC can be compared directly to PIC and FLIP.

Fu et al. 2017 Fu et al. extended this line of work with PolyPIC, which carries higher-order polynomial information on particles [FGG+17]. PolyPIC is useful context because it shows that APIC is not the endpoint of particle-grid transfer research, but one point in a broader effort to preserve more information across transfers.

Sancho et al. 2024 Sancho et al. later proposed the Impulse Particle-In-Cell method (iPIC), an APIC extension based on the impulse-gauge formulation of

the fluid equations [STBA24]. iPIC is useful recent context because it targets the preservation of circulation and vortical detail in smoke and liquid flows. Together, these methods motivate the thesis evaluation: transfer rules are compared not only by implementation structure, but by their effects on dissipation, detail preservation, and stability.

2.4 Material Point Methods

Material point methods are closely related to the hybrid particle-grid methods discussed above, but their main role is broader material simulation rather than only fluid velocity transfer. They combine Lagrangian material points with an Eulerian background grid: the material points carry state and history, while the grid provides a structured place for computation and interaction [SCS94; SSC+13]. This makes MPM relevant to the thesis because it uses the same general idea of exchanging information between particles and a grid. It is therefore an important additional evaluation setting because MPM-style material scenes can make stability differences between different transfer functions visible.

Sulsky et al. 1994 Sulsky et al. introduced a particle method for history-dependent materials [SCS94]. The paper is an important early reference for MPM because it connects particle-based material samples with grid-based computation in a setting where material history matters. For this Related Work chapter, the relevant point is not the detailed mechanics formulation, but the way MPM broadens particle-grid methods from fluid motion toward deformable, history-dependent materials.

Stomakhin et al. 2013 Stomakhin et al. brought MPM into computer graphics through snow simulation [SSC+13]. Their method uses a hybrid Eulerian/Lagrangian MPM together with an elastoplastic material model to handle snow behavior that can appear both solid-like and fluid-like. Compared with the earlier computational-mechanics framing, this work is important for the thesis because it shows particle-grid methods as practical tools for visually rich material animation. The constitutive model and deformation-gradient machinery are left outside the scope of this chapter, since the thesis comparison remains centered on transfer behavior rather than a full MPM formulation.

Chapter 3

Background

Fluid simulation is a numerical simulation problem: continuous motion has to be represented by discrete quantities, advanced in finite timesteps, and evaluated through the errors and artifacts introduced by those choices [Bri15]. Before PIC, FLIP, and APIC can be compared as transfer methods, the shared continuum model and the structure of the underlying particle-grid solver have to be established. This chapter therefore introduces the mathematical and numerical background used later in the Method and Evaluation chapters. We follow a common hybrid simulation cycle used by hybrid methods: particle state is transferred to the grid, grid-side forces and incompressibility operations are applied, updated information is transferred back to particles, and the particles are advected to the next timestep [Bri15; ZB05]. The detailed PIC, FLIP, and APIC transfer rules are left to the Method chapter. Here, the goal is to define the solver structure in which those rules operate.

3.1 Governing Fluid Equations

The numerical stages in this chapter are motivated by the continuum model for incompressible flow. For a fluid with constant density ρ, velocity field $\mathbf{u}$, pressure p, kinematic viscosity ν, and external acceleration $\mathbf{f}$, the incompressible Navier–Stokes equations can be written as

$$\frac{D\mathbf{u}}{Dt} = -\frac{1}{\rho}\nabla p + \nu\nabla^2\mathbf{u} + \mathbf{f}, \tag{3.1}$$

$$\nabla \cdot \mathbf{u} = 0. \tag{3.2}$$

Here, D/Dt denotes the material derivative, which follows the motion of the fluid rather than observing a fixed spatial point [Bri15; Wen08]. The first equation describes how velocity changes due to pressure, viscosity, and external forces. The

second equation states the incompressibility constraint: the velocity field should have zero divergence, so local fluid volume is preserved.

The hybrid solvers considered in this thesis do not solve these equations as one continuous system. Instead, they approximate the same physical structure with a timestep split into simpler numerical operations. Particles carry advected state through the domain, while the grid supplies the discrete operators for forces, boundaries, pressure, and incompressibility. The pressure term in Equation (3.1) becomes the projection step discussed later in this chapter, where a pressure field is found and its gradient is subtracted from the grid velocity. The divergence-free condition in Equation (3.2) provides the target condition for that projection. External accelerations such as gravity appear as grid-side force updates, and viscosity may be represented explicitly, implicitly, or omitted depending on the solver and application.

This thesis therefore uses the incompressible Navier–Stokes equations as the governing model behind the solver loop, but its central comparison is not a new discretization of every term in those equations. The main question is how PIC, FLIP, APIC, and related transfer rules move velocity information between particles and the grid, and how those choices affect stability, detail preservation, and ringing artifacts.

3.2 Particles and Grids

Hybrid particle-grid methods use two separate discretizations of the same simulated material [Bri15; ZB05]. Particles represent moving samples that carry information through the domain, while a grid provides a structured spatial representation on which numerical operations can be applied. This separation is useful because the two representations serve different purposes: particles follow material motion naturally, whereas grids make neighborhood relations, boundary conditions, and global operations such as pressure projection easier to express [Bri15]. The price of this combination is that information must be transferred between the two representations during every timestep.

In this thesis, a particle is a discrete computational sample, not a physically indivisible object. Its most fundamental property is its position, because all particle-grid exchange depends on where the sample lies inside the simulation domain. Additional particle state depends on the method and on the simulated quantity. For a fluid solver, particles commonly carry velocity or other advected material information [Bri15]. More specialized particle-grid methods enrich this state: APIC augments particles with local affine velocity information, while PolyPIC extends this idea with higher-order polynomial information [JSS+15; FGG+17]. These examples show that the particle is best understood as a carrier of method-

dependent state rather than as a fixed data structure with one universal set of attributes.

A grid, by contrast, decomposes the simulation domain into a regular set of cells and sample locations [Bri15]. Depending on the chosen discretization, quantities may be stored at cell centers, grid nodes, cell faces, or other locations associated with the grid. The placement of quantities is not merely a storage detail. It determines how discrete derivatives, interpolation, boundary conditions, and pressure projection are formulated [Bri15; HW+65]. For this reason, the following subsections distinguish collocated grids, where several quantities are stored at the same grid locations, from staggered grids, where different quantities are intentionally stored at different locations.

During a hybrid timestep, the grid acts as a temporary structured workspace. Particle quantities are first gathered onto the grid through particle-to-grid transfer [Bri15; JSS+15]. The solver then applies grid-side operations such as external forces, velocity extrapolation, boundary handling, and pressure projection. After these operations, the updated grid information is transferred back to the particles, and the particles are advected to their new positions [Bri15; ZB05]. Although the grid topology may persist in memory, the active grid values are reconstructed or updated from particle data as part of each timestep.

This structure explains why transfer quality is central to the methods studied in this thesis. Particles preserve moving state between timesteps, while the grid supplies the structured operators needed by the solver. Any mismatch between these representations can smooth information, discard unresolved detail, or preserve noisy particle-level modes [ZB05; JSS+15]. The later comparison of specific transfer methods is therefore a comparison of the information they share and drop between particles and grids.

3.2.1 Collocated Grid

A collocated grid stores several discrete quantities at the same spatial sample locations, such as cell centers or grid nodes [Bri15]. In a cell-centered collocated grid, each cell has one representative sample point, and fields such as mass, velocity, force, pressure, or material type can be indexed by the same cell coordinates. Figure 3.1 illustrates this layout for a two-dimensional cell-centered grid. This makes the layout visually and conceptually simple: the cell (i, j) denotes both a region of space and the location associated with the grid state for that region. For this reason, collocated layouts are a useful first grid representation before introducing more specialized placements. They are not merely a simplified example. Material point methods (MPM) commonly use a collocated background grid, often with mass, momentum, forces, and material-related quantities accumulated at shared grid nodes or cells [SCS94; SSC+13]. In the MPM settings considered here, the

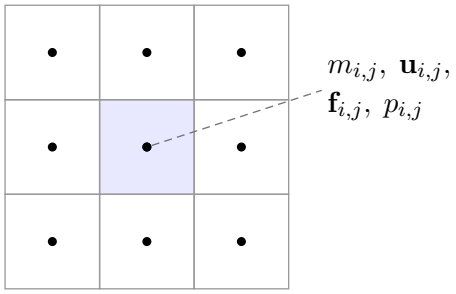


Figure 3.1: Schematic cell-centered collocated grid. Several grid quantities are associated with the same cell sample location, so the same index (i, j) can address mass, velocity, force, pressure, and other cell-centered state.

grid stage has a different role than the grid in an incompressible pressure-projection fluid solver. It acts mainly as a temporary workspace for accumulating particle state, applying forces, and computing updated grid velocities before information is transferred back to particles. It is not used to solve a pressure Poisson equation that couples pressure and velocity in order to enforce $\nabla \cdot \mathbf{u} = 0$. Therefore, the pressure-velocity decoupling issue that motivates staggered MAC grids in many incompressible fluid solvers is not applicable to these MPM-style particle-grid updates.

The advantages of collocated storage depend on the solver and the quantities being coupled. For incompressible fluid solvers, the relative placement of velocity and pressure affects the discrete divergence and pressure-gradient operators [Bri15]. If all quantities are stored at the same locations, some high-frequency modes can be poorly represented by the discrete operators [Bri15]. Staggered MAC grids address this by storing pressure and velocity components at different locations, which is why the next subsection introduces the staggered layout used by many grid-based fluid solvers [Bri15; HW+65].

3.2.2 Staggered MAC Grid

The collocated layout introduced in Section 3.2.1 stores quantities at shared sample locations. For the grid-based incompressible fluid solver considered in this chapter, however, this placement creates a difficulty in the pressure solve.

Enforcing incompressibility requires that the discrete velocity field satisfies $\nabla \cdot \mathbf{u} = 0$ everywhere. This is achieved by solving for a pressure field p and subtracting its gradient from the velocity. On a collocated grid, both p and $\mathbf{u}$ reside at cell centers, so the discrete divergence and gradient operators reach across two cells rather than one. As a consequence, a high-frequency checkerboard pattern of alternating pressure values can have a vanishing discrete gradient on this two-cell

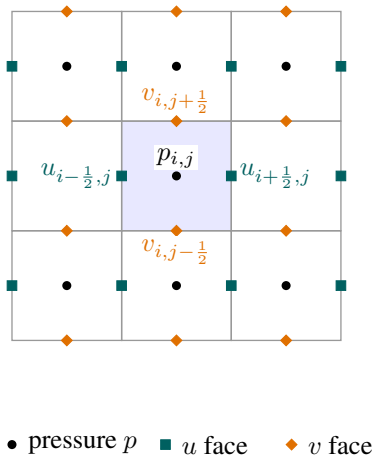


Figure 3.2: Schematic MAC grid layout in two dimensions. Pressure is stored at cell centers, while each velocity component is stored at the center of the face it is normal to. The half-index notation reflects the spatial offset between the pressure and velocity sample locations.

stencil. Such a mode produces no velocity correction and is therefore invisible to the pressure solve [Bri15]. Pressure and velocity are effectively decoupled at the grid scale, and these spurious modes can grow unchecked.

The Marker-and-Cell (MAC) grid resolves this by placing each scalar pressure value at the cell center and each velocity component at the center of the corresponding cell face [HW+65]. Figure 3.2 illustrates this layout. In two dimensions, the horizontal component u is stored on the left and right faces of each cell, and the vertical component v on the top and bottom faces. Using a half-index convention, the face-centered samples are written as

$$u_{i+\frac{1}{2},j}, \qquad v_{i,j+\frac{1}{2}}, \tag{3.3}$$

where $u_{i+\frac{1}{2},j}$ denotes the horizontal velocity on the face between cells (i,j) and $(i{+}1,j)$. Similarly, $v_{i,j+\frac{1}{2}}$ denotes the vertical velocity on the face between cells (i,j) and $(i,j{+}1)$.

This placement tightens the discrete operators that couple pressure and velocity. The discrete divergence at cell (i,j) is computed directly from the four surrounding face velocities,

$$(\nabla \cdot \mathbf{u})_{i,j} = \frac{u_{i+\frac{1}{2},j} - u_{i-\frac{1}{2},j}}{\Delta x} + \frac{v_{i,j+\frac{1}{2}} - v_{i,j-\frac{1}{2}}}{\Delta x}, \tag{3.4}$$

where Δx is the uniform cell width. Each term is a difference between immediately adjacent face samples, so the stencil spans only a single cell. Figure 3.3 illustrates which samples contribute to this operator.

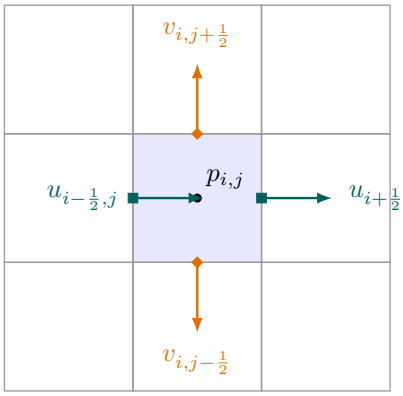


Figure 3.3: Discrete divergence stencil on a MAC grid. The divergence at cell (i, j) depends only on the four immediately surrounding face velocities, giving a compact one-cell stencil that tightly couples velocity and pressure.

Symmetrically, the discrete pressure gradient that corrects the velocity field after the pressure solve acts directly on the same face samples,

$$u_{i+\frac{1}{2},j} \leftarrow u_{i+\frac{1}{2},j} - \frac{\Delta t}{\rho} \frac{p_{i+1,j} - p_{i,j}}{\Delta x}, \tag{3.5}$$

$$v_{i,j+\frac{1}{2}} \leftarrow v_{i,j+\frac{1}{2}} - \frac{\Delta t}{\rho} \frac{p_{i,j+1} - p_{i,j}}{\Delta x}, \tag{3.6}$$

where Δt is the time step and ρ is the fluid density. Each face velocity is updated by the pressure difference across the single cell boundary it sits on. Pressure and velocity are therefore spatially interlocked: a checkerboard pressure mode would produce nonzero differences at every face and is no longer invisible to the discrete operators [Bri15]. This tight coupling is the defining numerical advantage of the staggered layout.

3.3 Particle To Grid

After particles and grid storage locations have been defined, the first solver-stage operation is to construct grid quantities from particle data. For the fluid solver considered in this thesis, the most important quantity transferred in this stage is velocity as it is used by all transfer functions we discuss. Particles carry velocities at continuously varying positions, while the pressure solve, boundary handling, force application, and extrapolation stages operate on the staggered MAC grid. Particle-to-grid transfer therefore converts the particle velocity samples into face-centered grid velocities that can be used by the following grid-side operations [Bri15].

The transfer is local: each particle contributes only to nearby grid samples, with interpolation weights determined by the particle position relative to those samples. For one spatial coordinate, the linear hat kernel used in the implementation can be written as

$$N(r) = \max(0, 1 - |r|), \tag{3.7}$$

where r is the distance between a particle position and a grid sample, normalized by the grid spacing. In two dimensions, the weight for a particle at $\mathbf{x}_p = (x_p, y_p)$ and a grid sample at $\mathbf{x}_i = (x_i, y_i)$ is the tensor product

$$w_{ip} = N\left(\frac{x_p - x_i}{\Delta x}\right) N\left(\frac{y_p - y_i}{\Delta x}\right). \tag{3.8}$$

The three-dimensional case adds the corresponding factor in the third coordinate direction.

For a collocated grid, all quantities share the same sample position $\mathbf{x}_i$, so the same weights serve every field at that location. Particle mass and velocity can therefore be accumulated in a single pass,

$$m_i = \sum_p w_{ip} m_p, \tag{3.9}$$

with velocity computed as the mass-weighted average

$$\mathbf{u}_i = \frac{\sum_p w_{ip} m_p \mathbf{u}_p}{m_i}, \tag{3.10}$$

where $\mathbf{u}_p$ and m_p are the particle velocity and mass. Because all quantities share the same target positions, no component-wise distinction is needed. Figure 3.4 shows this for a particle and its neighboring collocated grid samples.

The staggered layout makes this transfer component-specific, as Figure 3.5 illustrates. The horizontal velocity component is accumulated at u-face samples, while the vertical velocity component is accumulated at v-face samples. In three dimensions, the same idea extends to the third velocity component. Because the face locations are offset from each other, each component evaluates its interpolation weights relative to its own grid of face samples. The solver therefore maintains separate accumulated weights for each component and normalizes the accumulated values after all particle contributions have been processed. In the implementation used for this thesis, particles have equal mass for this transfer, so the mass factors in Equation (3.10) reduce to a normalization by the accumulated interpolation weights. For one velocity component, the transferred value is therefore computed as

$$u_i = \frac{\sum_p w_{ip} u_p}{\sum_p w_{ip}}. \tag{3.11}$$

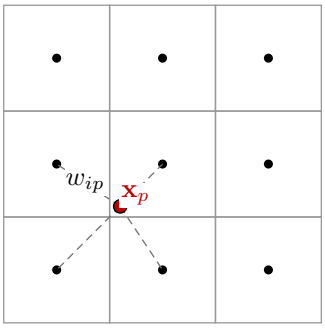


Figure 3.4: Particle-to-grid transfer on a collocated grid. A particle contributes to nearby grid samples according to interpolation weights. Because the relevant quantities share the same sample locations, the same target positions can be used when accumulating mass, velocity, and other collocated grid state.

This equation is evaluated separately for the face samples of each velocity component.

The particle-to-grid stage also connects the moving particle representation to the grid's cell classification. Cells containing particles are marked as fluid cells, which allows later grid-side stages to distinguish fluid regions from air or solid regions. After transfer, the solver has a face-centered grid velocity field and a fluid-cell classification that can be used by force application, velocity extrapolation, boundary handling, and pressure projection. Grid faces with $\sum_p w_{ip} = 0$ receive no particle contribution; in the implementation, such samples are left at their cleared value and are handled during the velocity-extrapolation and boundary stages.

MPM uses the same broad transfer idea. The collocated transfer shown in Figure 3.4 captures the part relevant here: particle state is accumulated on shared grid sample locations. The stress and force computations specific to material point methods are separate from this background discussion.

This transfer step is not merely a data-copy operation. It is a resampling step from moving particle samples to a structured grid. As a result, it determines which particle-scale information is visible to the subsequent grid solve and which variation is averaged away. This makes particle-to-grid transfer an important part of the numerical behavior studied later, even before the thesis introduces the specific transfer rules compared in the Method chapter. After the grid-side update is complete, grid-to-particle transfer uses the same geometric relation in the opposite direction to sample updated grid information back to particle positions. The details of that return transfer are introduced in Section 3.8.

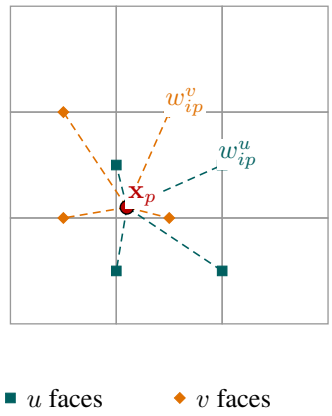


Figure 3.5: Particle-to-grid transfer on a staggered grid. Particle velocity information is accumulated at the face locations where the corresponding MAC velocity components are stored. Horizontal and vertical velocity components are therefore transferred to different sets of grid samples.

3.4 Apply Forces

After particle-to-grid transfer, the solver has a velocity field on the grid and a classification of which cells contain fluid. External forces can then be applied directly to the grid velocities before the later extrapolation, boundary, and pressure-projection stages. For a force represented as an acceleration, this stage adds a time-step-scaled velocity change to the affected grid samples.

A simple example is gravity. With y increasing upward, $g < 0$ denotes the vertical gravitational acceleration, and the vertical velocity component is updated as

$$v \leftarrow v + \Delta t\, g, \tag{3.12}$$

On a MAC grid, this update is applied to the vertical face velocities, because those samples store the vertical component of the velocity field. Only faces adjacent to at least one fluid cell are affected. This keeps the force update tied to the active fluid region instead of modifying unrelated empty grid samples.

Gravity is only one possible force in this stage. The same structure can be used for additional accelerations or force fields, for example localized stirring, wind-like forcing, scripted impulses, or rotational fields used to create vortex-like motion in controlled experiments. After forces have been applied, the velocity field may still need extrapolation near unsupported faces, boundary handling near solid cells, and pressure projection to enforce the incompressibility constraint.

3.5 Extrapolate Velocities

After force application, the grid velocity field is still only reliable at samples that are supported by the active fluid region. Particle-to-grid transfer assigns velocities where particles contribute, and the force stage modifies the corresponding active samples. However, later solver stages may require velocity values in a small neighborhood around the fluid, for example near the free surface or next to cells that will be used during interpolation. Grid-based fluid solvers therefore commonly extrapolate known velocity values into nearby unsupported samples before continuing with the timestep [Bri15].

For a staggered MAC grid, this operation has to respect the storage location of each velocity component. A horizontal face velocity is treated as valid when the face touches a fluid cell on either side, while a vertical face velocity is valid when it touches a fluid cell above or below. In three dimensions, the same idea applies to the third component on its own face grid. Because these component grids are shifted relative to each other, validity is tracked separately for each component.

The extrapolation itself can be understood as a layer-by-layer fill operation. Initially valid face samples form the first layer. Neighboring unsupported samples are assigned to the next layer, and additional layers are discovered by moving outward across the component grid. When a new face sample is filled, its value is computed as the average of neighboring samples that already belong to a closer layer. This propagates nearby fluid velocities into a narrow band without creating new particle information or changing the fluid-cell classification.

The result is not a pressure projection and does not enforce a boundary condition by itself. It only supplies usable velocity values near the active fluid region so that the following boundary, pressure, interpolation, and advection operations do not have to sample completely unsupported grid values. In the solver sequence considered here, this is useful both before boundary handling and pressure projection, and again after projection before information is transferred back to the particles.

3.6 Enforce Boundaries

After velocity extrapolation, the grid may contain velocity values on faces close to both fluid and solid cells. These values are useful for interpolation and for the following solver stages, but they must still respect the solid boundaries of the domain. The solver therefore uses the cell classification to identify faces that lie next to solid cells and constrains the corresponding velocity components before the pressure projection is applied.

On a MAC grid, this constraint can be expressed directly on the face velocities. A horizontal face velocity stores motion through a vertical cell face. If either cell

adjacent to that face is solid, a nonzero horizontal velocity would represent motion through the solid region, so the horizontal component at that face is set to zero. Likewise, a vertical face velocity is set to zero when either the cell below or the cell above the face is solid. In three dimensions, the same rule extends to the third component: each face velocity is tested against the two cells separated by that face.

This operation implements a simple stationary-wall condition for the grid velocity field: fluid is prevented from passing through solid cells. It does not make the velocity field incompressible by itself, and it does not handle particle motion or particle-wall collisions. Those responsibilities belong to the pressure-projection and advection stages. Instead, boundary enforcement prepares the extrapolated grid velocity field so that the subsequent pressure solve starts from velocities that already respect the solid parts of the domain [Bri15].

3.7 Pressure Solve And Projection

After boundary enforcement, the grid velocity field respects the stationary solid cells of the domain, but it is not necessarily incompressible. For an incompressible fluid, each fluid cell should have no net local volume change. The velocity passing through each face, called flux, should balance between the faces where fluid enters and the faces where it leaves. In differential notation this condition is written as $\nabla \cdot \mathbf{u} = 0$. On the MAC grid, the same idea is measured by the face-centered divergence stencil introduced in Equation (3.4). A positive imbalance corresponds to a locally expanding cell, while a negative imbalance corresponds to local compression. Figure 3.6 illustrates this interpretation before the pressure equation is introduced.

The pressure projection step removes this divergent part of the intermediate grid velocity. Let $\mathbf{u}^*$ denote the velocity after particle to grid transfer, force application, extrapolation, and boundary handling. Projection computes a pressure field and subtracts its gradient from this intermediate velocity, following the standard grid-based incompressible flow construction described by Bridson [Bri15]:

$$\mathbf{u}^{n+1} = \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla p. \tag{3.13}$$

For the staggered grid used here, the component-wise version of this update has already been given in Equation (3.6). The important point is that pressure is not transferred to particles directly at this stage. Instead, pressure changes the grid face velocities, and particles receive the corrected motion later during grid to particle transfer.

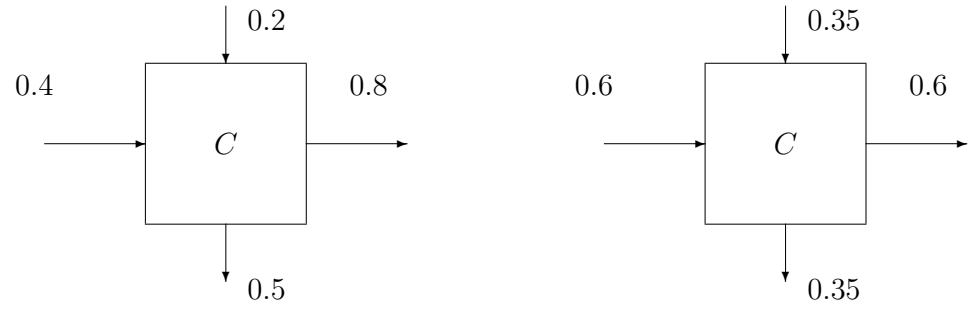


Figure 3.6: Projection goal on one MAC cell. Before projection (left), the outgoing face fluxes exceed the incoming ones: $(0.8 - 0.4) + (0.5 - 0.2) = 0.7 > 0$. After projection (right), the fluxes balance: $(0.6 - 0.6) + (0.35 - 0.35) = 0$. The numbers are illustrative magnitudes only.

The pressure equation follows from requiring the projected velocity to be divergence-free. Substituting the pressure update into $\nabla \cdot \mathbf{u}^{n+1} = 0$ gives

$$\nabla \cdot \mathbf{u}^* - \frac{\Delta t}{\rho} \nabla \cdot \nabla p = 0, \tag{3.14}$$

and therefore, for constant density,

$$\nabla^2 p = \frac{\rho}{\Delta t} \nabla \cdot \mathbf{u}^*. \tag{3.15}$$

This is the pressure Poisson equation used by the projection method [Bri15].

Discretizing the pressure equation on the MAC grid gives one equation for each fluid cell, following a common projection construction for grid-based incompressible fluid solvers [Bri15]. Let $C = (i, j)$ be a fluid cell, and let $\mathcal{F}(C)$ denote the fluid neighbors of C among its four axis-aligned neighbors. The implementation uses the negative-Laplacian sign convention

$$\frac{a_C p_C - \sum_{N \in \mathcal{F}(C)} p_N}{\Delta x^2} = \frac{\rho}{\Delta t} \left(-d_C^* + s_C^* \right), \tag{3.16}$$

where

$$d_C^* = \frac{u^*_{i+\frac{1}{2},j} - u^*_{i-\frac{1}{2},j} + v^*_{i,j+\frac{1}{2}} - v^*_{i,j-\frac{1}{2}}}{\Delta x} \tag{3.17}$$

is the discrete divergence of the intermediate velocity. The coefficient a_C counts non-solid neighbors of C. Fluid neighbors contribute pressure unknowns to the sum, air neighbors contribute only to a_C and are treated as zero-pressure free-surface cells, and solid neighbors contribute no pressure unknown across the wall.

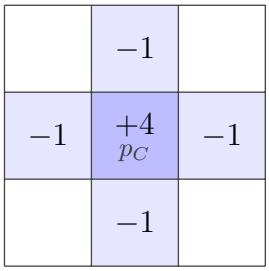


Figure 3.7: Five-point pressure stencil for an interior fluid cell. The discretised pressure equation (Equation (3.16)) places $a_C = 4$ on the diagonal, acting on the cell's own pressure p_C, and a coefficient -1 on each of the four axis-aligned neighbors. At the fluid boundary the row changes: an air neighbor drops its -1 but still counts toward a_C as a zero-pressure cell, while a solid neighbor is excluded from both.

For an interior fluid cell surrounded by fluid, Equation (3.16) is the familiar five-point Laplacian stencil, with $a_C = 4$ on the cell's own pressure and a coefficient of -1 for each of its four neighbors, as shown in Figure 3.7. The term s_C^* collects solid-wall velocity corrections. For the stationary boundaries used here, the wall velocity is zero, and these corrections vanish when the boundary enforcement stage has already set the corresponding face velocities to zero.

Collecting these per-cell equations for all fluid cells gives the sparse linear system

$$A\mathbf{p} = \mathbf{b}, \tag{3.18}$$

where $\mathbf{p}$ collects the unknown pressure values and $\mathbf{b}$ is built from the divergence of $\mathbf{u}^*$ together with boundary contributions. These boundary conditions are necessary because the pressure equation otherwise would not define what happens at the edge of the fluid domain.

The resulting pressure system is large and sparse, so it is solved iteratively rather than by explicitly inverting the matrix. A common choice for symmetric pressure systems of this kind is the conjugate gradient method, which starts from an initial pressure estimate and iteratively reduces the residual of $A\mathbf{p} = \mathbf{b}$ until a convergence tolerance is met [Bri15]. The method is well suited to this problem because it only requires matrix-vector products rather than explicit matrix operations, and those products parallelize efficiently across the grid structure. When the initial estimate is far from the solution convergence can be slow, so a preconditioner is applied to transform the system into an equivalent form that converges faster. Several preconditioners are applicable here, ranging from a simple diagonal Jacobi preconditioner to more effective options such as MIC(0), a modified incomplete Cholesky variant that Bridson recommends as particularly well suited to the sparse pressure stencil [Bri15]. Regardless of the chosen preconditioner, the solve requires

a residual tolerance and usually an iteration cap to bound runtime on difficult timesteps.

Once the pressure values have been found, the projection updates the MAC face velocities using pressure differences across faces. Faces adjacent to solid cells remain constrained by the boundary treatment, while faces between fluid and non-solid cells receive the pressure-gradient correction. This reduces the divergence of fluid cells and produces the grid velocity field used by the remaining solver stages. Because the projection changes the reliable fluid-region velocities but does not automatically fill every nearby unsupported face sample, velocity extrapolation is applied again afterward. This second extrapolation extends the corrected grid velocity into the narrow band needed for interpolation and makes the following grid to particle transfer less abrupt near free surfaces.

3.8 Grid To Particle

After pressure projection and the second velocity extrapolation, the grid contains the corrected velocity field that will drive the next particle motion. The projection has modified the MAC face velocities through pressure forces, and extrapolation has extended these corrected values into the nearby region that may be sampled by particles. The grid to particle stage now transfers this updated grid information back to the moving particle representation before particle advection. This is the return direction of the transfer described in Section 3.3: particle to grid transfer constructs grid values for the solver, while the grid to particle transfer samples the result back to the particles [Bri15].

The interpolation uses the same geometric weights introduced in Equation (3.7) and Equation (3.8). For a generic grid quantity q_i, the value sampled at particle p can be written as

$$q_{p,\mathrm{grid}} = \sum_i w_{ip} q_i, \tag{3.19}$$

where the sum runs over the neighboring grid samples in the interpolation stencil. In contrast to particle to grid transfer, this operation does not accumulate many particles into one grid sample. It evaluates one particle position from nearby grid values. For a complete interior stencil, the interpolation weights form a convex combination of the neighboring samples [Bri15].

On a MAC grid, the velocity components are interpolated from their own staggered face grids. In two dimensions, the horizontal component is sampled from nearby u faces and the vertical component is sampled from nearby v faces:

$$u_{p,\mathrm{grid}} = \sum_i w^u_{ip} u_i, \qquad v_{p,\mathrm{grid}} = \sum_i w^v_{ip} v_i. \tag{3.20}$$

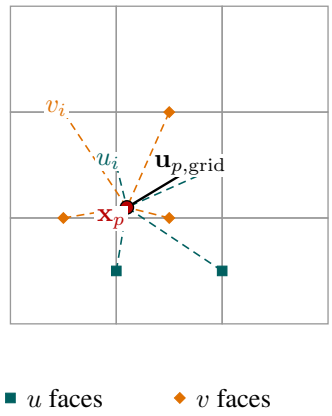


Figure 3.8: Grid-to-particle transfer on a staggered grid. The particle samples each velocity component from the MAC face grid where that component is stored. The horizontal and vertical components therefore use different neighboring face samples, even though both use the same interpolation kernel.

The superscripts on the weights indicate that each component evaluates the same interpolation kernel relative to different face-sample locations. In three dimensions, the third component is handled analogously on the w face grid. Figure 3.8 illustrates this component-wise sampling for one particle.

Different transfer methods use the sampled grid information differently. The simplest example is PIC, where the particle velocity is replaced by the velocity interpolated from the grid,

$$\mathbf{u}_p^{n+1} = \mathbf{u}_{p,\text{grid}}. \tag{3.21}$$

Other transfer methods return different information to the particles, which changes how much previous particle motion and local variation remain after the grid solve. Those method-specific choices are the subject of the Method chapter. For the Background chapter, the important point is that grid to particle transfer samples the corrected grid velocity back onto the particles so that the following advection stage moves them with the updated flow.

3.9 Advect

Once the grid velocity has been transferred back to the particles, the solver has updated particle velocities but not yet updated particle positions. Advection completes the timestep by moving each particle according to its velocity. For a particle at position $\mathbf{x}_p$, the ideal continuous motion is described by

$$\frac{\mathrm{d}\mathbf{x}_p}{\mathrm{d}t} = \mathbf{u}_p, \tag{3.22}$$

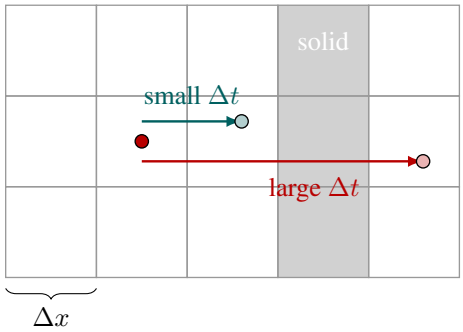


Figure 3.9: CFL intuition for particle advection. A short timestep keeps particle motion local relative to the grid, so boundary crossings can be detected near the wall. A timestep that is too large can move a particle across several cells at once and may skip over a solid boundary between checks.

which in practice is integrated numerically over the discrete timestep.

A simple forward Euler step evaluates the velocity only at the beginning of the timestep, but curved motion is followed more accurately with a higher-order method. The midpoint method is a compact second-order Runge-Kutta update:

$$\mathbf{x}_{p,\mathrm{mid}} = \mathbf{x}_p^n + \frac{\Delta t}{2}\mathbf{u}(\mathbf{x}_p^n, t^n), \tag{3.23}$$

$$\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \Delta t\, \mathbf{u}(\mathbf{x}_{p,\mathrm{mid}}, t^n + \tfrac{1}{2}\Delta t). \tag{3.24}$$

The first sample estimates a midpoint position, and the second sample moves the particle with a velocity evaluated near the middle of the step. This is close to the practical FLIP-style advection described by Zhu and Bridson, who move particles through the grid velocity field with a second-order Runge-Kutta solver and CFL-limited substeps [ZB05].

The timestep used for advection cannot be chosen independently of the grid resolution and the velocity magnitude. A common nondimensional measure is the CFL number,

$$C_{\mathrm{CFL}} = \frac{\|\mathbf{u}\|\Delta t}{\Delta x}, \tag{3.25}$$

which compares the distance traveled during one timestep to the grid spacing. If this number is large, a particle can cross too much grid structure in one update. This can make interpolation less local, can cause boundary checks to miss thin features, and can allow a particle to pass through a wall between two sampled positions. Figure 3.9 illustrates this situation for a simple grid boundary.

A typical timestep restriction can be written as

$$\Delta t \le C_{\max}\frac{\Delta x}{\|\mathbf{u}\|_{\max}}, \tag{3.26}$$

where $C_{\max}$ is a chosen safety target and $\|\mathbf{u}\|_{\max}$ is the maximum relevant speed. Practical solvers enforce such a restriction by choosing a smaller timestep, splitting a frame step into several solver substeps, or adapting the number of substeps when the flow becomes faster. This does not remove all numerical error, but it keeps particle motion local enough for interpolation, boundary handling, and grid classification to remain meaningful [Bri15; Wen08].

Boundary handling is still needed during advection. The pressure solve and boundary velocity enforcement constrain the grid velocity field, but a particle trajectory is a geometric path through a discrete domain. Because of interpolation error, finite timesteps, and grid-level boundary representation, a trial particle position can still end up inside a solid cell. When that happens, the advection stage should correct the particle back to the fluid side of the boundary. Depending on the material and boundary model, the response may reject the step, project the particle out of the solid, or modify the velocity component normal to the wall.

With advection complete, the shared timestep described in this chapter is closed. Figure 3.10 summarizes the implemented order, including the two velocity-extrapolation passes. With this solver loop in place, the Method chapter can focus on the central comparison of this thesis, namely how PIC, FLIP, and APIC transfer different information between particles and the grid.

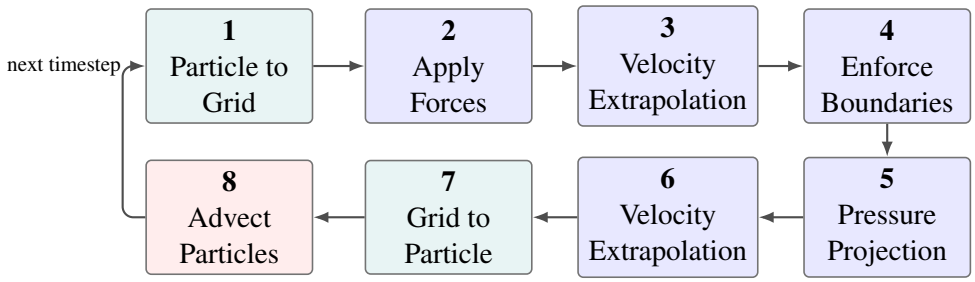


Figure 3.10: Implemented fluid-solver step order shared by the two- and three-dimensional configurations. The two extrapolation passes occur before boundary enforcement and after pressure projection.

Chapter 4

Method

The Background chapter established a single solver cycle shared by every hybrid method considered here. Particle state is transferred to the grid, the grid-side forces and incompressibility operations are applied, the result is transferred back to the particles, and the particles are advected to the next step. That chapter stopped short of the transfer rules themselves and used particle-in-cell (PIC) only as the simplest grid-to-particle example for completeness. This chapter closes the remaining gap by making precise what distinguishes PIC, fluid-implicit-particle (FLIP), affine particle-in-cell (APIC), and the polynomial extension PolyPIC within that one cycle.

The guiding idea is that the transfer step is the single component that varies between the methods while every other stage of the solver is held fixed. Treating the transfer rule as the only independent variable turns an informal comparison into a controlled one and frames the rest of the thesis. Beyond defining the methods, the chapter establishes how they are analysed, namely which numerical and physical properties matter and which metrics make those properties measurable rather than merely visible. It also sets out why a purpose-built solver, rather than an off-the-shelf tool, is needed to observe and compare this internal behaviour under identical conditions. The concrete test scenes and the measured results that apply these criteria follow in the Evaluation chapter.

4.1 Particle-in-Cell (PIC)

PIC is one of the earliest hybrid particle–grid schemes. The particles hold the state of the simulation, while the grid serves only as a temporary scaffold on which the spatial operators are evaluated. A particle carries only a position and a velocity, together with a fixed mass, and stores nothing about how the velocity varies in its neighbourhood [Har62].

In the particle-to-grid step each particle scatters its mass and momentum to the nearby grid nodes using the interpolation weights of the Background chapter, recall Equation (3.7) and Equation (3.8), whose partition of unity is the property the method leans on,

$$w_{ip} = N(\mathbf{x}_i - \mathbf{x}_p), \qquad \sum_i w_{ip} = 1. \tag{4.1}$$

The grid velocity is recovered as the mass-weighted average of the contributing particles,

$$m_i = \sum_p w_{ip}\, m_p, \qquad \mathbf{u}_i = \frac{1}{m_i} \sum_p w_{ip}\, m_p\, \mathbf{v}_p. \tag{4.2}$$

For the equal-mass fluid particles used here m_p cancels and the transfer reduces to a plain weighted average,

$$\mathbf{u}_i = \frac{\sum_p w_{ip}\, \mathbf{v}_p}{\sum_p w_{ip}}. \tag{4.3}$$

On the staggered MAC grid these transfers are applied per velocity component, each on its own faces, as described in the Background chapter. The grid-side stages introduced there, force integration, boundary enforcement, and pressure projection, then act on this grid velocity, and PIC makes no assumption about them.

The defining step is the grid-to-particle transfer. Each particle velocity is replaced outright by the updated grid velocity interpolated at its position,

$$\mathbf{v}_p^{n+1} = \sum_i w_{ip}\, \tilde{\mathbf{u}}_i, \tag{4.4}$$

where $\tilde{\mathbf{u}}_i$ is the grid velocity after the force and projection stages, so the particle's previous velocity is discarded entirely. The particle is then advected with this freshly interpolated velocity using the midpoint scheme of the Background chapter, recall Equation (3.24), and the cycle repeats. A PIC particle therefore keeps no memory of its own motion, since its velocity is rebuilt from the grid on every timestep [Bri15].

Because the velocity is resampled twice per step, once when scattering to the grid and once when gathering back, each timestep acts as a low-pass filter on the velocity field [Bri15]. The smoothing follows directly from the transfer equations. Since the weights sum to one, the gather in Equation (4.4) reproduces a spatially constant velocity exactly, but any sub-cell variation is averaged away on the round-trip through Equation (4.2), as sketched in Figure 4.1. The result is strong numerical dissipation that resembles an artificial viscosity, removing kinetic energy and washing out small-scale detail. Because the whole field is reconstructed every step, this dissipation accumulates rather than staying bounded. In practice

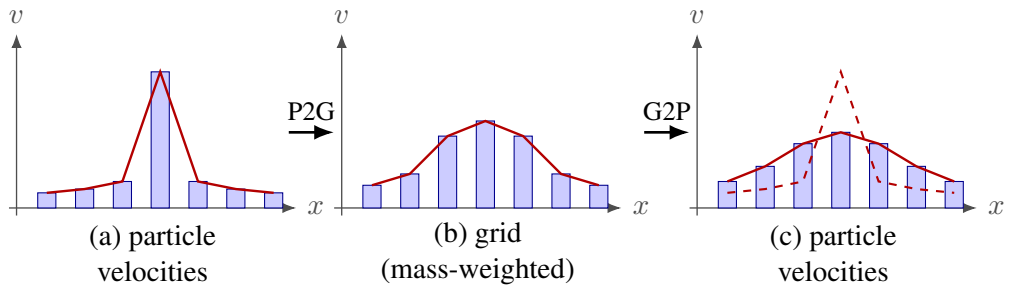


Figure 4.1: Schematic of the smoothing introduced by a single PIC transfer round-trip along one spatial direction; bar height represents velocity magnitude at each sample. A sharp particle velocity profile (a) is averaged onto the grid (b) and gathered back to the particles (c). The dashed line in (c) repeats the original profile from (a): the peak is flattened and spread, illustrating the dissipative low-pass character of the transfer. The figure is illustrative and not to scale.

PIC is very stable and robust, damping noise and oscillations, at the cost of overly smooth motion in which vortices and sharp features decay quickly.

Sub-cell velocity variation such as local rotation and shear cannot be represented by a single particle velocity and is lost in the averaging, so rotational motion and angular momentum are poorly preserved [JSS+15]. Refining the grid or raising the particles-per-cell count reduces this dissipation but does not remove it. PIC thus anchors the dissipative end of the transfer-method spectrum and provides the baseline that motivates FLIP, APIC, and PolyPIC in the sections that follow.

4.2 Fluid-Implicit-Particle (FLIP)

FLIP was developed as a direct response to the excessive dissipation of PIC, while keeping the same picture of particles which carry the state and the grid as a temporary scaffold [BR86]. It was later popularised for graphics by Zhu and Bridson [ZB05]. A FLIP particle stores exactly the same data as a PIC particle, namely a position, a velocity, and a fixed mass, with no additional neighbourhood information.

The particle-to-grid step is identical to PIC, and the grid-side stages from the Background chapter are likewise unchanged. FLIP differs from PIC in a single place, which is the grid-to-particle step. Instead of overwriting the particle velocity with the interpolated grid velocity, FLIP transfers back only the change in grid velocity produced by the grid solve. Concretely, the grid velocity after the solve is compared with the grid velocity just after the particle-to-grid scatter, and the

interpolated difference is added to the particle's existing velocity,

$$\mathbf{v}_p^{n+1} = \mathbf{v}_p^{n} + \sum_i w_{ip} \left(\tilde{\mathbf{u}}_i - \mathbf{u}_i \right). \tag{4.5}$$

This requires retaining the pre-solve grid velocity $\mathbf{u}_i$ so that the per-face increment can be formed before the gather. Consequently the particle keeps its own velocity from previous steps and the grid only nudges it. Unlike PIC, a FLIP particle has velocity memory.

The relationship to PIC is clearer once the update is split apart. Writing the PIC reconstruction of the post-solve field as a gather,

$$\mathbf{v}_p^{\text{PIC}} = \sum_i w_{ip} \, \tilde{\mathbf{u}}_i, \tag{4.6}$$

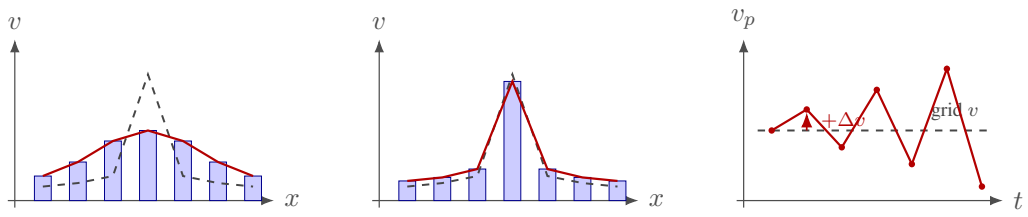
Equation (4.5) separates into a PIC term and a correction,

$$\mathbf{v}_p^{\text{FLIP}} = \mathbf{v}_p^{\text{PIC}} + \left(\mathbf{v}_p^{n} - \sum_i w_{ip} \, \mathbf{u}_i \right). \tag{4.7}$$

The bracketed term is the particle's retained velocity minus the PIC reconstruction of the pre-solve field, which is exactly the detail that PIC would have discarded. Because only the increment passes through the interpolation round-trip, the bulk of each particle's velocity is never repeatedly averaged, so FLIP is far less dissipative and preserves kinetic energy and small-scale detail much better than PIC [Bri15].

The trade-off is that the particle velocities are no longer tied to a single smooth grid field, and each particle instead accumulates its own history. The particle representation therefore contains more degrees of freedom than the grid can directly represent, and the FLIP update does not remove particle-scale discrepancies that are lost during the particle-to-grid transfer [JST17]. This explains how such discrepancies can persist, but it is distinct from the effect of the particle count. Increasing the number of particles per cell increases the number of particle-space degrees of freedom not directly representable on the grid, whereas low particle counts make the transfer itself more poorly sampled. In the latter case, noisy grid increments are generated more easily and are then retained by the FLIP update instead of being filtered out by a PIC-style overwrite. The resulting particle-level velocity noise can drive oscillations, ringing, and a clear loss of stability relative to PIC, particularly on under-resolved grids [JST17]. Figure 4.2 illustrates both the preservation of a sharp velocity profile and the temporal accumulation of unfiltered increments.

In practice pure FLIP ($\alpha = 1$) is rarely used on its own, because its particle noise can make simulations unstable. Solvers almost universally blend the two



(a) PIC: full replacement (b) FLIP: velocity increment (c) accumulating increments

Figure 4.2: Effect of one grid-to-particle transfer on a sharp velocity profile (dashed: profile entering the step; solid: profile after transfer). PIC (a) rebuilds the particle velocities from the smoothed grid field and flattens the peak, the source of its strong dissipation. FLIP (b) adds only the interpolated grid *increment* to the retained particle velocities, so the bulk of the profile bypasses the smoothing and the detail is largely preserved. The same mechanism is destabilising over time (c), which tracks the velocity v_p of a single particle across successive steps t. Each step adds the locally interpolated grid increment Δv on top of the velocity the particle already carries. Because these increments are not filtered by the round-trip, the particle's velocity is free to drift away from the grid field it should follow, and the unconstrained mode grows step by step into an oscillation of increasing amplitude. The accumulation is temporal, not spatial: no summation across neighbouring nodes is implied. Illustrative and not to scale.

updates with a parameter $\alpha \in [0, 1]$,

$$\mathbf{v}_p^{n+1} = (1-\alpha)\,\mathbf{v}_p^{\text{PIC}} + \alpha\,\mathbf{v}_p^{\text{FLIP}} = \sum_i w_{ip}\,\tilde{\mathbf{u}}_i + \alpha\Big(\mathbf{v}_p^n - \sum_i w_{ip}\,\mathbf{u}_i\Big), \tag{4.8}$$

with α close to one, keeping most of FLIP's detail while bleeding in just enough of PIC's damping to control the noise [ZB05; Bri15]. The value of α is a reported parameter, and pure FLIP is retained mainly as the noisiest reference point. It is worth noting that even pure FLIP is not the identity transfer: in general $\sum_i w_{ip}\,\mathbf{u}_i \neq \mathbf{v}_p^n$, because gathering the pre-solve field reconstructs the smoothed particle velocity rather than the original, so the correction in Equation (4.7) removes precisely the round-trip smoothing that makes PIC dissipative. The blend in Equation (4.8) is the form usually implemented, since only $\mathbf{u}_i$ and $\tilde{\mathbf{u}}_i$ are needed.

FLIP therefore retains detail that PIC loses, but it does so without a principled treatment of sub-cell rotation and at the cost of stability, which motivates APIC in the following section [JSS+15].

4.3 Affine Particle-in-Cell (APIC)

APIC was introduced to retain the stability of PIC while reducing its dissipation through an enriched particle state rather than the velocity memory used by

FLIP [JSS+15]. In addition to its velocity, each particle carries an affine velocity matrix $\mathbf{C}_p$ that describes the local linear variation of the velocity. The particle therefore represents the field

$$\mathbf{v}(\mathbf{x}) \approx \mathbf{v}_p + \mathbf{C}_p(\mathbf{x} - \mathbf{x}_p) \tag{4.9}$$

in its neighbourhood instead of representing only a constant velocity.

The affine matrix can be expressed formally through a moment matrix $\mathbf{B}_p$ and the symmetric, inertia-like tensor $\mathbf{D}_p$,

$$\mathbf{C}_p = \mathbf{B}_p\,\mathbf{D}_p^{-1}, \qquad \mathbf{D}_p = \sum_i w_{ip}\,(\mathbf{x}_i - \mathbf{x}_p)(\mathbf{x}_i - \mathbf{x}_p)^\top. \tag{4.10}$$

For the multilinear interpolation used in this thesis, $\mathbf{D}_p$ depends on the particle's position within its cell and is not reduced to a constant scaled identity. It is neither formed nor inverted in the implementation. Because the interpolation weight gradients reconstruct the velocity gradient directly, the affine matrix is instead obtained as

$$\mathbf{C}_p = \sum_i \mathbf{u}_i\,(\nabla w_{ip})^\top, \tag{4.11}$$

the gradient of the interpolated velocity. This form remains well defined when a particle lies on a cell facet, where $\mathbf{D}_p$ would be singular [JST17].

During the particle-to-grid step, each particle scatters both its velocity and the local linear variation evaluated at the receiving node,

$$m_i = \sum_p w_{ip}\,m_p, \qquad \mathbf{u}_i = \frac{1}{m_i}\sum_p w_{ip}\,m_p\big(\mathbf{v}_p + \mathbf{C}_p(\mathbf{x}_i - \mathbf{x}_p)\big). \tag{4.12}$$

Compared with the PIC transfer in Equation (4.2), the bracketed affine term reinjects the sub-cell variation that constant particle velocities cannot carry. The transfer is constructed to conserve both total linear and total angular momentum [JSS+15; JST17].

The grid-to-particle step fully reconstructs the particle velocity from the post-solve grid velocity, exactly as in PIC, and then re-estimates the affine state for the next transfer,

$$\mathbf{v}_p^{n+1} = \sum_i w_{ip}\,\tilde{\mathbf{u}}_i, \tag{4.13}$$

$$\mathbf{B}_p^{n+1} = \sum_i w_{ip}\,\tilde{\mathbf{u}}_i\,(\mathbf{x}_i - \mathbf{x}_p)^\top, \qquad \mathbf{C}_p^{n+1} = \mathbf{B}_p^{n+1}\,\mathbf{D}_p^{-1}. \tag{4.14}$$

The affine update in Equation (4.14) is evaluated through the reconstruction of Equation (4.11). The velocity update in Equation (4.13) is identical to the PIC gather in Equation (4.4), so APIC keeps no FLIP-style velocity memory.

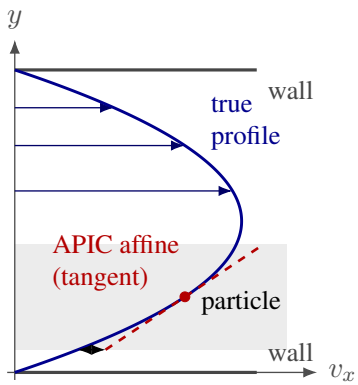


Figure 4.3: A curved velocity profile that the affine model linearises away. A parabolic channel-flow profile (blue: v_x across the channel, fast in the middle, zero at the walls) is sampled by a particle (red) in the lower half. APIC stores the local velocity and its slope, giving the straight tangent line (dashed); within the particle's cell (shaded band) this affine approximation departs from the true curve, so the curvature is lost. Polynomial extensions such as PolyPIC can restore these higher-order modes. Illustrative, not to scale.

Because the particle velocity is rebuilt from the grid after every solve, subgrid discrepancies are filtered instead of being retained and accumulated. APIC therefore inherits the stability of PIC and avoids the ringing and noise growth associated with FLIP. At the same time, the affine term carries local rotation and shear across the transfer instead of averaging them away, which makes APIC substantially less dissipative than PIC. Setting $\mathbf{C}_p = \mathbf{0}$ in Equation (4.12) recovers the PIC particle-to-grid transfer exactly. The defining advantage of APIC is that it combines this filtering with the conservation of both linear and angular momentum across the transfer cycle [JSS+15; JST17]. This requires additional per-particle state. In d spatial dimensions, the affine matrix stores d^2 entries and increases both memory use and transfer work, which provides the basis for the later memory comparison. APIC represents velocity variation only up to affine order within a cell. It preserves the local velocity and its slope, but not its curvature. For example, a parabolic channel-flow profile is approximated locally by a straight tangent line, so its bend within the cell is lost. Figure 4.3 illustrates this limitation, which motivates polynomial extensions such as PolyPIC [FGG+17].

APIC therefore occupies the middle ground between the preceding methods. It retains the grid-based filtering of PIC, preserves local linear detail without the velocity memory of FLIP, and provides the central transfer method evaluated in this thesis. Its remaining first-order limitation leads directly to PolyPIC.

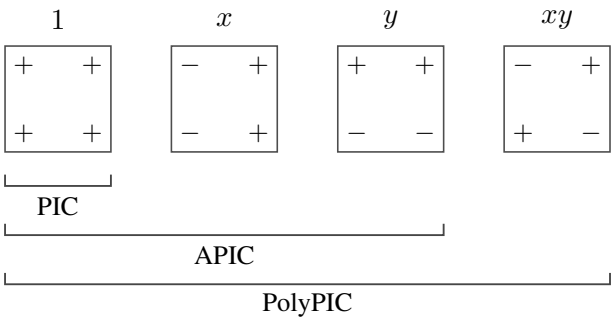


Figure 4.4: The bilinear modes a particle can carry. Each square shows the sign pattern of one multilinear basis function over a cell: the constant mode 1, the two linear modes x and y, and the cross mode xy. PIC carries only the constant mode, APIC adds the two linear modes (the affine matrix), and PolyPIC adds the cross mode, completing the space the bilinear grid can represent. The three-dimensional case adds the analogous z, xz, yz, and xyz modes.

4.4 Polynomial Particle-in-Cell (PolyPIC)

PolyPIC generalises APIC by matching the particle representation to the function space of the interpolation kernel [FGG+17]. Instead of storing only a velocity and its local affine variation, each particle stores one coefficient per polynomial mode for every velocity component. Its local velocity field is represented as

$$\mathbf{v}(\mathbf{x}) \approx \sum_r \mathbf{c}_{p,r}\, P_r(\mathbf{x} - \mathbf{x}_p), \qquad \{P_r\} = \{1,\ x,\ y,\ xy\}\ \text{(2D)}, \tag{4.15}$$

with the eight trilinear modes

$$\{1,\ x,\ y,\ z,\ xy,\ xz,\ yz,\ xyz\}$$

in three dimensions. These are exactly the multilinear functions represented by the grid. The constant coefficient is the particle velocity, and the linear coefficients form the affine matrix $\mathbf{C}_p$. Consequently, PIC is obtained by retaining only the constant mode, while APIC retains the constant and linear modes. PolyPIC additionally carries the cross-modes that complete the grid-representable space.

The particle-to-grid step evaluates the stored polynomial at each receiving node and scatters the result with the interpolation weights,

$$m_i = \sum_p w_{ip}\, m_p, \qquad \mathbf{u}_i = \frac{1}{m_i} \sum_p w_{ip}\, m_p \sum_r \mathbf{c}_{p,r}\, P_r(\mathbf{x}_i - \mathbf{x}_p). \tag{4.16}$$

This has the same structure as the affine scatter in Equation (4.12), extended to the additional multilinear modes. Setting the cross-mode coefficients to zero recovers the APIC particle-to-grid transfer exactly.

During the grid-to-particle step, each coefficient is recovered by projecting the post-solve grid velocity onto its mode. For a general basis, the moment system contains

$$M_{rs} = \sum_i w_{ip}\, P_r P_s, \qquad \mathbf{b}_r = \sum_i w_{ip}\, P_r\, \tilde{\mathbf{u}}_i.$$

With the particle-centred tensor-product basis used by PolyPIC, the multilinear modes are mutually orthogonal under the interpolation-weighted inner product, so the moment system is diagonal. Each coefficient is therefore obtained independently as

$$\mathbf{c}_{p,r} = \frac{\sum_i w_{ip}\, P_r(\mathbf{x}_i - \mathbf{x}_p)\, \tilde{\mathbf{u}}_i}{\sum_i w_{ip}\, P_r(\mathbf{x}_i - \mathbf{x}_p)^2}. \tag{4.17}$$

No coupled matrix inversion is required because each mode uses only its scalar normalization denominator [FGG+17]. Retaining only the constant coefficient reduces Equation (4.17) to the PIC velocity update in Equation (4.4). Retaining the constant and linear coefficients gives APIC.

Within this grid-representable space, the PolyPIC transfer is non-dissipative up to advection and grid-side operations [FGG+17]. This is the essential difference to the affine truncation used by APIC, where cross-modes such as xy are absent from the APIC particle state and are therefore filtered by the round-trip transfer, while PolyPIC retains them and therefore avoids the transfer-induced loss of the corresponding grid-resolved velocity variation.

Like APIC, PolyPIC reconstructs the complete particle state from the grid after each solve and keeps no FLIP-style velocity memory. Modes outside the grid-representable space are filtered instead of accumulating on the particles. PolyPIC therefore retains the stability of PIC and APIC, while its constant and affine sub-modes preserve the same linear and angular momentum properties as APIC [JST17; FGG+17]. PolyPIC is therefore least dissipative for grid-representable velocity content that lies outside the affine subspace, while reducing to the APIC representation when no cross-mode content is present.

The improvement over APIC is limited by the interpolation kernel. Under bilinear interpolation, the only additional two-dimensional mode is the cross term xy. Axis-aligned curvature terms such as x^2 and y^2 cannot be represented by the grid and therefore cannot be recovered by the transfer. Capturing such higher-order structure would require a higher-order interpolation kernel and a correspondingly larger particle basis [FGG+17].

Completing the multilinear basis requires additional per-particle coefficients beyond the affine matrix, which increases both memory use and transfer work. PolyPIC thus completes the progression from PIC through APIC: it matches the full grid-representable space and makes the transfer information-preserving within that space, at the highest per-particle cost of the four methods [FGG+17].

Table 4.1: Qualitative comparison of the four transfer methods. All four share the solver cycle and conserve linear momentum. They differ in the per-particle state, in how velocity is returned from the grid, and in the resulting numerical behaviour. Entries are qualitative and are examined quantitatively in the Evaluation chapter.

	PIC	FLIP	APIC	PolyPIC
Per-particle state	velocity	velocity	velocity + $\mathbf{C}_p$	velocity + $\mathbf{C}_p$+ cross-modes
Velocity update (G2P)	replacement	increment	replacement	replacement
Numerical dissipation	high	low	low	lowest
Stability / noise	high / damped	low / ringing-prone	high	high
Angular momentum	not preserved	not guaranteed	conserved	conserved
Relative memory cost	low	low	higher	highest

4.5 Comparison of Transfer Methods

The four transfer methods use the same solver cycle and the same mass-weighted particle-to-grid structure. All four conserve linear momentum. Their differences lie in the state carried by each particle and in the way velocity is returned from the grid, as summarised in Table 4.1.

PIC and FLIP occupy opposite ends of the central transfer trade-off. PIC rebuilds particle velocities from the grid and is stable but strongly dissipative. FLIP retains particle velocity history and preserves more detail, but this same memory allows particle-scale discrepancies to persist and makes the method prone to noise and ringing. APIC addresses this trade-off by enriching the particle state instead of changing how much velocity is transferred. Its affine field restores local linear variation, preserves angular momentum, and retains the filtering provided by a full grid-to-particle reconstruction.

PolyPIC extends the same enrichment to the complete multilinear basis represented by the grid. The additional cross-modes retain information that the affine APIC state omits, making the transfer information-preserving within the grid-representable space [FGG+17]. This further reduces transfer-induced dissipation, but requires additional per-particle coefficients and therefore the highest memory cost of the four methods.

These properties lead to the hypotheses examined in the Evaluation chapter. Relative to PIC, APIC is expected to preserve substantially more energy and small-scale detail. Relative to FLIP, it is expected to avoid the growth of particle noise and ringing. It is also expected to conserve angular momentum where neither baseline provides the same guarantee, at the cost of additional per-particle memory. PolyPIC is expected to preserve still more grid-representable detail than APIC because it retains the cross-modes that the affine representation discards [FGG+17].

The lower dissipation of PolyPIC may also leave less damping for errors introduced by particle advection, pressure projection, or interpolation. The evalua-

tion therefore tests both whether PolyPIC preserves more detail and whether this reduced damping makes it more sensitive in practice. These statements are expectations rather than results. The following sections define the solver configuration and evaluation criteria used to test them.

4.6 Material Point Method

The preceding transfer comparison is not limited to incompressible fluids. The material point method (MPM) follows the same broad particle-grid structure established in the Background chapter. Material points carry mass, momentum, position, and other advected state. This state is transferred to a temporary background grid, updated there, and transferred back before the points move. For deformable materials, the particles additionally carry deformation and material history. A constitutive model uses this state to determine the stress forces applied during the grid update. Stomakhin et al. combine these components in their MPM formulation for snow [SSC+13].

The role of the grid has already been introduced in Section 3.2.1. For the present comparison, the important connection is that MPM exposes the same two transfer interfaces as the fluid solver. The transfer rule determines how particle momentum and the local particle velocity representation are mapped to the grid, and how the updated grid velocity is represented on the particles again. The material update changes what is computed between these transfers, but it does not remove the transfer problem studied in this thesis. The same distinction between constant, affine, and polynomial particle representations therefore carries over to MPM.

The interpolation order determines how many of these polynomial modes can participate in the exchange. The two-dimensional fluid solver uses bilinear interpolation on each component grid of the staggered MAC layout. Its local transfer space contains the four modes $\{1, x, y, xy\}$ discussed in Section 4.4. This is the complete PolyPIC basis for that interpolation. Terms containing x^2 or y^2 cannot be recovered from the bilinear stencil, so adding them to the fluid particle state would introduce modes that the grid cannot represent.

The MPM setting considered in this thesis instead uses tensor-product quadratic B-spline interpolation on a collocated grid. A particle then interacts with three grid nodes per coordinate rather than two. In two dimensions, the resulting 3×3 stencil supports a nine-mode PolyPIC representation. In addition to the four multilinear modes, this representation contains five mass-orthogonal modes with a quadratic factor [FGG+17]. These modes are applicable because the higher-order interpolation supplies the additional local grid samples needed to determine them.

The difference therefore follows from the interpolation order and support, not from the collocated placement alone.

This section only needs the part of MPM that connects the fluid transfer comparison to a deformable-material particle-grid method. A complete MPM simulation also requires the evolution of material state and a constitutive law. Plasticity, hardening, and the detailed snow model are therefore outside the scope of the transfer discussion here. They are treated in the original MPM snow formulation by Stomakhin et al. [SSC+13].

4.7 A Custom Solver for Controlled Analysis

The transfer methods could in principle be compared within an existing production package. Such packages provide mature hybrid solvers that are more feature-complete than a purpose-built research implementation can be in the context of this thesis. However, they are designed to produce complete simulation results rather than controlled transfer experiments. The transfer step is embedded in a larger pipeline that may include particle reseeding, adaptive timestepping, vorticity confinement, viscosity and damping options, surface-tension models, and tuned boundary treatments. Each component can alter dissipation, noise, or stability independently, which makes it difficult to attribute an observed effect to the transfer method alone. Stabilisation may mask or imitate the behaviour under study, particularly when it cannot be fully disabled or inspected.

Limited access to solver internals introduces a second source of uncertainty. Without complete source access and documentation, the exact computation performed by each stage, the order of those stages, and any corrections applied between them cannot be verified. Production tools also tend to expose the state only at the end of a frame, whereas identifying the source of transfer behaviour requires observations between the individual stages of the hybrid cycle. Runtime and memory measurements from a heavily optimised black box would likewise reflect implementation-specific optimisations as much as the intrinsic cost of each transfer method.

All experiments therefore use a purpose-built C++ solver: The Evaluation Platform for Particle-in-Cell Methods (EPIC).[1] It implements the hybrid cycle described in the Background chapter in both two-dimensional and three-dimensional configurations. The transfer method is selected at runtime, while forces, velocity extrapolation, boundary enforcement, pressure projection, and particle advection use shared code. For the PIC/FLIP family, the blend parameter α from Equation (4.8) selects pure PIC at $\alpha = 0$, a blended update for $0 < \alpha < 1$, or pure FLIP

[1] `https://github.com/TorbenSteegmann/EPIC` — the thesis version is tagged `thesis`.

at $\alpha = 1$. The affine and polynomial methods change only the particle state and the transfer routines required to carry their additional modes. Keeping every other stage unchanged isolates the transfer method as the independent variable.

The implementation was checked through a combination of transfer-specific tests, solver diagnostics, and visual comparison. The transfer routines were verified on simple fields whose expected behaviour is known. Additionally PolyPIC reduces to the PIC result when only the constant mode is active, to the APIC result when only the constant and affine modes are active, and reproduces the additional multilinear cross-mode when it is enabled. This provides a direct consistency check across the implemented transfer methods. The remaining solver stages were checked through standard numerical diagnostics, including mass conservation, pre- and post-projection divergence, pressure-solver residuals, CFL numbers, and the absence of invalid or out-of-bounds particles in controlled scenes. In addition, simple test cases were visually compared against publicly available fluid-simulation references to ensure that the qualitative behaviour of the solver was plausible before using it for quantitative transfer-method comparisons.

The solver records kinetic energy and maximum speed after particle-to-grid transfer, force application, extrapolation, boundary enforcement, pressure projection, grid-to-particle transfer, and advection. This stage-level instrumentation allows a change in energy to be assigned to a particular part of the cycle rather than only to the timestep as a whole. Each step also exports total mass, linear momentum, centre of mass, kinetic energy, potential energy, total mechanical energy, and a fluid-area estimate used as a volume proxy. Angular momentum is part of the required diagnostic set because it directly tests the conservation property that distinguishes APIC and PolyPIC from the baseline methods.

Incompressibility and solver health are tracked through divergence before and after projection, pressure-solver iteration counts and residuals, and the CFL number together with the particle responsible for its maximum value. Counts of invalid particles, out-of-bounds particles, and boundary collisions expose failures that aggregate energy or momentum values may not reveal. A dedicated ringing measure is recorded from the end-of-step grid residual, the part of each particle velocity that the grid cannot reconstruct, as defined in Section 4.8. Its kinetic energy, normalised by a fixed scene reference energy, exposes sustained particle-scale oscillation as a sustained non-zero value instead of relying only on visual inspection.

Runtime is measured through scoped profiling in the deterministic headless runner. The same runner writes all measurements to CSV, allowing an experiment to be reproduced from its scene definition and transfer-mode settings without manual interaction.

An interactive front end provides particle inspection and per-stage visualisation during scene design and debugging. While it can display and export frame data, it

does not provide the reported measurements used in this thesis. All quantitative results originate from the reproducible headless execution path. This includes the renderings of the scenes in Chapter 5 which are created via a short python script from raw particle data exported during the run.

4.8 Evaluation Criteria and Metrics

The preceding sections defined four transfer methods and a solver in which they can be exchanged while the remaining simulation stages stay fixed. A meaningful comparison now requires a measurable definition of transfer quality. Visual plausibility is necessary, but it is not sufficient as a scientific criterion. Visual judgement is subjective, difficult to reproduce, and unsuitable for comparisons across parameter studies. Two observers may rank the same simulations differently, and a verdict that an animation looks plausible cannot be repeated or stated with error bounds.

The relevant failure modes may also remain hidden in rendered output. Excessive dissipation can produce smooth and visually pleasing motion while steadily removing energy and rotational detail. Sub-grid velocity noise may remain beneath an apparently calm surface before emerging as visible ringing. A single frame reveals little about these accumulation effects. Gradual energy loss, growing noise, and volume drift become visible only in time series.

Physical quantities provide objective reference points without requiring an exact solution of the full flow. Mass, linear momentum, angular momentum, energy, and volume should either be conserved or follow known behaviour, so their measured drift quantifies errors introduced by the simulation and its transfer stages. Where no exact invariant exists, as for noise and ringing, the criterion must instead be defined explicitly from the simulation state. The same numerical indicator can then be compared across methods, scenes, and parameter settings.

Not every recorded quantity is used as a comparative result. Some metrics serve primarily as validity checks: they are monitored in every run, but only discussed further if they deviate from their expected behaviour. Total particle mass is the clearest example. Since the experiments use fixed-mass particles, preserved mass confirms that no particles were lost or invalidated, while mass drift indicates a failed configuration rather than a meaningful transfer-method difference.

The evaluation criteria are divided into four groups. Conservation and physical fidelity measure the preservation of physical quantities. Transfer quality and artifacts describe dissipation and ringing, with ringing as the primary quality criterion of this thesis. Robustness and sensitivity measure how behaviour changes with discretisation parameters. Computational cost records runtime and memory

requirements. Together these criteria turn the expectations in Section 4.5 into testable predictions for the Evaluation chapter.

4.8.1 Conservation and Physical Fidelity

The first group of criteria uses mechanical quantities with known physical behaviour as the measuring instrument. Their recorded totals can be assessed without a reference simulation. External forces, boundary interactions, and pressure projection may change these quantities for physical or discretisation-related reasons. Across the transfer stages, mass and linear momentum should remain unchanged, while changes in particle-centre angular momentum and energy reveal the macroscopic conservation and dissipation behaviour of the selected transfer method.

Total particle mass is

$$M = \sum_p m_p. \tag{4.18}$$

It remains constant by construction while every particle survives the timestep. Because the experiments use a single fluid density and fixed particle masses, changes in total mass correspond to changes in the valid particle count. Mass is therefore interpreted together with the recorded numbers of invalid and out-of-bounds particles, which distinguish particle loss from transfer arithmetic.

Total linear momentum is

$$\mathbf{P} = \sum_p m_p \mathbf{v}_p. \tag{4.19}$$

The mass-weighted particle-to-grid scatter preserves this quantity by assembling grid momentum from the sum of the particle contributions [JSS+15]. Under gravity alone and in the absence of boundary impulses, its expected evolution is

$$\mathbf{P}(t) = \mathbf{P}(0) + M\mathbf{g}t. \tag{4.20}$$

Residual deviation from this evolution measures momentum created or destroyed by the discretisation.

Following the decomposition of particle angular momentum into particle-centre and internal affine contributions used for APIC [JSS+15; JST17], the orbital contribution about the centre of mass is

$$\mathbf{L}_{\text{orb}} = \sum_p m_p(\mathbf{x}_p - \mathbf{x}_c) \times \mathbf{v}_p. \tag{4.21}$$

This quantity measures the macroscopic rotational motion visible in the particle-centre velocities. It does not include the internal angular-momentum contribution

carried by the affine state of APIC and the affine sub-modes of PolyPIC. Testing their theoretical angular-momentum conservation property directly therefore requires the corresponding affine contribution [JSS+15; JST17]. In a scene without external torque, retention of the particle-centre contribution is measured by

$$R_{L,\mathrm{orb}}(t) = \frac{\mathbf{L}_{\mathrm{orb}}(t) \cdot \mathbf{L}_{\mathrm{orb}}(0)}{\|\mathbf{L}_{\mathrm{orb}}(0)\|^2}. \tag{4.22}$$

This signed measure equals one when the initial particle-centre angular momentum is preserved, decreases as macroscopic rotational motion is lost, and exposes a reversal of its direction.

For a flow whose rotation is concentrated in a compact region, the core-concentrated angular momentum restricts the orbital sum to the particles within a fixed radius R of the centre of mass,

$$L_z^{\mathrm{core}} = \sum_{\|\mathbf{x}_p - \mathbf{x}_c\| \le R} m_p \left[(\mathbf{x}_p - \mathbf{x}_c) \times \mathbf{v}_p\right]_z. \tag{4.23}$$

Its retention $L_z^{\mathrm{core}}(t)/L_z^{\mathrm{core}}(0)$ isolates how much of the rotation initially held in the core remains concentrated there, which a transfer that diffuses the swirl outward reduces even when the total angular momentum is preserved.

The particle kinetic and gravitational potential energies are

$$E_k = \frac{1}{2}\sum_p m_p \|\mathbf{v}_p\|^2, \qquad E_p = -\sum_p m_p\, \mathbf{g} \cdot \mathbf{x}_p, \qquad E = E_k + E_p. \tag{4.24}$$

The normalised total energy $E(t)/E(0)$ allows decay to be compared across scenes and resolutions. A per-step total cannot by itself identify the source of that decay because the pressure projection and boundary stage may also remove kinetic energy. The solver therefore records kinetic energy after every stage. Differences across particle-to-grid and grid-to-particle transfer measure transfer-induced dissipation, while the remaining differences separate force, boundary, and projection contributions.

Fluid volume is estimated from the Eulerian classification as

$$V = N_{\mathrm{fluid}}\, \Delta x^d, \qquad \frac{V(t)}{V(0)}, \tag{4.25}$$

where N_{fluid} is the number of fluid-classified cells and d is the spatial dimension. The projection enforces a divergence-free grid field, so sustained volume drift is interpreted together with divergence measured before and after projection. These recordings distinguish a pressure solve that fails to remove divergence from transfer or advection that reintroduces inconsistency after the solve.

4.8.2 Transfer Quality and Artifacts

Conserved totals do not reveal every transfer error. Particle-to-particle velocity oscillations can cancel in global sums while the local field remains strongly disordered. The quality criteria therefore examine the structure and temporal variation of the velocity field in addition to its total momentum and energy.

After particle-to-grid transfer, the grid contains only the part of the particle velocity field represented at the grid resolution. Let $E^n_{k,p}$ denote the kinetic energy of the particle-centre velocities before this transfer and $E^n_{k,g}$ the grid kinetic energy immediately afterwards. Their difference is

$$E^n_{\mathrm{sub}} = E^n_{k,p} - E^n_{k,g}. \tag{4.26}$$

This difference measures particle-centre kinetic energy absent from the scattered grid field. For APIC and PolyPIC it does not include energy stored only in the affine or polynomial particle modes, so it is not a complete account of their subgrid content. Both energies are recorded at the transfer boundary, allowing E_{sub} to be tracked over time. Sustained growth indicates that velocity variation is accumulating in the particle-centre velocities outside the grid-representable field.

The stage recordings also provide a loss budget for one transfer cycle. Let

$$\Delta E^n_{\mathrm{P2G}} = E^n_{k,g} - E^n_{k,p}, \qquad \Delta E^n_{\mathrm{G2P}} = E^{n+1}_{k,p} - \widetilde{E}^n_{k,g}, \tag{4.27}$$

where $\widetilde{E}^n_{k,g}$ is the kinetic energy of the post-solve grid field immediately before grid-to-particle transfer. Their sum,

$$\Delta E^n_{\mathrm{transfer}} = \Delta E^n_{\mathrm{P2G}} + \Delta E^n_{\mathrm{G2P}}, \tag{4.28}$$

isolates the combined kinetic-energy change across the two transfer stages of one timestep, while the intervening force, boundary, and projection contributions are measured separately. A persistently negative value indicates repeated smoothing. A positive contribution during grid-to-particle transfer indicates energy appearing in particle modes that was not present in the gathered grid field.

The temporal counterpart to the energy measures is the ringing metric. Ringing is a finite-grid instability of particle-grid transfer rather than a feature of one method, and it can arise across the PIC family. It was characterised for low-speed particle-in-cell flow [Bra88], and later APIC work interprets related artifacts in terms of particle-carried modes that are poorly represented, or lie in the null space, of the grid transfer [JST17]. FLIP is especially prone to retaining these modes because it updates particle velocities incrementally from grid changes rather than overwriting them with a grid reconstruction. Modes that are only weakly represented on the grid are not directly corrected by grid-space operations such as

pressure projection and boundary enforcement, so they can persist on the particles after the bulk motion has decayed.

The metric quantifies these modes as a particle-grid reconstruction error. The current particle velocities are scattered to the grid to form the field $\mathbf{u}^n$, which is then interpolated back to the particles with the same grid-to-particle interpolation. The residual is the difference between a particle's original velocity and this reconstructed grid velocity,

$$\mathbf{r}_p^n = \mathbf{v}_p^n - \mathcal{I}_h \mathbf{u}^n(\mathbf{x}_p^n), \tag{4.29}$$

and its kinetic energy over all N particles is

$$E_{\text{res}}^n = \frac{1}{2} \sum_p m_p \left\| \mathbf{v}_p^n - \mathcal{I}_h \mathbf{u}^n(\mathbf{x}_p^n) \right\|^2. \tag{4.30}$$

This is an energy of particle-grid inconsistency, the unresolved particle-centre velocity energy that the grid reconstruction does not reproduce. It is a diagnostic quantity rather than a separate conserved energy of the system. Because PIC reconstructs each particle velocity from the grid, it tends to reduce the residual. The measured value need not be exactly zero, since the measurement stage, advection, interpolation asymmetry, boundary handling, and numerical error all contribute. FLIP can instead retain residual modes on the particles, allowing them to persist or accumulate. Coherent motion is reproduced by the grid and contributes little, so the residual reflects retained sub-grid modes rather than legitimate flow.

For comparison across methods the residual energy is normalised. A per-step residual fraction

$$f_{\text{res}}^n = \frac{E_{\text{res}}^n}{E_{k,p}^n + \varepsilon} \tag{4.31}$$

relates it to the particle kinetic energy $E_{k,p}^n$ at the same stage, with $\varepsilon = 10^{-12}$. It serves only as a secondary diagnostic, because the denominator becomes small as the flow approaches rest and the fraction then grows unstable. The headline normalisation instead uses a fixed reference energy, the largest particle kinetic energy reached by any method m at any step n of the scene,

$$E_{\text{ref}} = \max_{m,n} E_{k,p,m}^n, \qquad e^n = \frac{E_{\text{res}}^n + \varepsilon}{E_{\text{ref}}}. \tag{4.32}$$

Because every method shares the same E_{ref} within a scene, e^n preserves the absolute differences in residual energy between methods. The additive ε keeps the normalised value positive when the residual approaches zero.

The reported ringing score is the geometric mean of e^n over a fixed late-time evaluation window W, taken after the bulk motion has decayed,

$$\bar{e} = \exp\left(\frac{1}{|W|} \sum_{n \in W} \log(e^n) \right), \tag{4.33}$$

The geometric mean is used because e^n ranges over several orders of magnitude across methods, a spread on which an arithmetic mean would report only the largest values. The main summary figure reports $\bar{e}$ as the scalar ringing score, and because all methods share the same E_{ref} it preserves the absolute differences in residual energy rather than reporting a per-method fraction. A summary of the absolute residual energy over the same window is taken as a companion, confirming that the normalised score reflects the actual residual magnitude and not only the behaviour of the normalisation.

The score is computed over the same fixed evaluation window for every method. The recorded boundary-collision and projection-correction series are inspected only to interpret isolated spikes in the residual, and are not used to remove steps from the score. The sub-grid energy E_{sub} of Equation (4.26) measures the related reservoir at the particle-to-grid boundary from which this residual is expected to grow.

Rendered comparisons may additionally colour particles by local rotation to expose sub-grid disorder spatially. This visualisation complements the quantitative series but is itself not used as a numerical criterion.

4.8.3 Robustness and Sensitivity

The preceding criteria describe transfer quality at a selected operating point. Robustness instead asks how that quality changes when the discretisation becomes less favourable. A method that behaves well only at high resolution and small timesteps is less useful in practice than one whose errors grow gradually as these conditions are relaxed.

Robustness is measured through controlled parameter sweeps rather than through a separate scalar metric. One discretisation parameter is varied while the scene, deterministic particle seeding, and all remaining parameters are held fixed. For every setting and transfer method, the conservation, energy, sub-grid energy, ringing, and solver-health series defined in the preceding subsections are recorded again. The resulting curves show both the quality at each operating point and the rate at which that quality degrades.

Outright failure requires no additional derived measure. The solver records non-finite and out-of-bounds particle counts at every step, and a run is classified as diverged when either count becomes non-zero. The first parameter value at which this occurs provides an empirical stability boundary for the method. Maximum speed and CFL history show how the run approaches this boundary.

Boundary and free-surface behaviour are interpreted from the same diagnostics. Boundary-collision counts and the velocity corrections applied at solid walls show whether degradation is concentrated near the domain boundary. The diagnostic associated with the particle producing the maximum CFL number includes its

distance to the nearest boundary, revealing whether extreme velocities originate systematically in the wall region. Free-surface behaviour is examined in scenes where the bulk motion decays toward rest. In such cases, persistent late-time ringing or particle activity is interpreted together with the rendered particle data to determine whether the remaining motion is concentrated near the sparsely sampled surface region.

The timestep sweep varies a fixed Δt while the grid and particle seeding remain unchanged. Adaptive timestep selection is disabled so that every method uses the same fixed timestep at each sweep setting. Each run is reported against its recorded CFL number rather than against Δt alone, which expresses the timestep relative to the grid spacing and velocity scale.

The grid-resolution sweep changes the cell size while preserving the physical domain and simulated duration. A deviation that decreases under refinement is treated as discretisation error. A deviation that persists or grows at matched CFL indicates behaviour associated with the transfer rule rather than only with grid resolution.

Every value in each sweep is evaluated for PIC, FLIP, APIC, and PolyPIC from identical initial states. Neighbouring runs therefore differ in one controlled variable, while corresponding runs across methods differ only in the selected transfer rule.

4.8.4 Computational Cost

Computational cost turns the preceding quality criteria into trade-offs. Runtime is measured under the same controlled conditions as the physical and numerical diagnostics, while memory is compared through analytic per-particle state counts. All timings are obtained from the headless runner without rendering or interactive input, using one fixed machine and build configuration for every method.

Wall time is recorded for each simulation step and for every stage of the solver cycle through scoped profiling. This separation is necessary because the stages scale differently. Transfer work depends primarily on the particle count and interpolation stencil, while pressure projection depends on the grid size and the number of solver iterations. The cost of a transfer method is therefore reported as the combined time spent in its particle-to-grid and grid-to-particle stages, together with the total step time. The shared grid-side stages provide a common baseline. Per-stage timing also makes the result easier to interpret across scenes and resolutions, where the fraction of a timestep spent in transfer rather than projection may differ substantially.

The particle-state memory is treated analytically rather than as a measured process-memory quantity. Since all transfer methods are implemented within the same solver, the allocated memory also reflects shared infrastructure, inactive fields, alignment, and implementation choices that are not intrinsic to the methods.

A measured total would therefore not provide a fair comparison of the transfer rules themselves.

The shared particle position contributes d scalars to every method and is therefore not method-dependent. The transfer-dependent velocity state consists of one velocity vector for PIC and FLIP, the velocity together with the $d \times d$ affine matrix for APIC, and one coefficient for every velocity component and every multilinear mode for PolyPIC. Since the tensor-product multilinear basis contains 2^d modes, the transfer-dependent state is

$$N_{\text{transfer}}^{\text{PIC}} = N_{\text{transfer}}^{\text{FLIP}} = d, \qquad N_{\text{transfer}}^{\text{APIC}} = d + d^2, \qquad N_{\text{transfer}}^{\text{PolyPIC}} = d\,2^d. \quad (4.34)$$

Including positional scalars, this gives 4, 8, and 10 total particle-state scalars in two dimensions, and 6, 15, and 27 in three dimensions.

Chapter 5

Evaluation

The previous chapter defined four transfer methods, a solver in which only the transfer step varies, and the criteria by which transfer quality is judged. It set out the hypothesis that APIC retains PIC's stability while approaching FLIP's detail and conserving angular momentum, and that PolyPIC carries the same enrichment further to a transfer that preserves every mode the grid can represent [FGG+17]. This chapter puts that hypothesis to the test.

Each criterion responds most clearly to a scene built to provoke it, so the experiments are organised as a catalogue of controlled scenes rather than a single demonstration. Some isolate one quantity in the cleanest possible setting, while others combine every effect in the violent, boundary-dominated regime the methods are used for in practice. Within each controlled comparison, PIC, FLIP, APIC, and PolyPIC are run from the same initial state through the deterministic headless solver with all non-transfer settings held fixed, so that each reported difference can be traced to the transfer rule rather than to a difference in setup, tuning, or machine.

The analysis is carried out primarily in two dimensions, where transfer artifacts are easiest to isolate, visualise, and resolve at high resolutions, and where parameter studies are cheap enough to run densely. Two dimensions are not, however, the whole story. Free-surface fragmentation, the relative size of the sub-grid noise reservoir, and the runtime and state cost all change in three dimensions. A dedicated section therefore re-examines the central findings in a three-dimensional dam break to establish how far the two-dimensional conclusions carry over.

The chapter proceeds from the protocol common to all runs, through the scene catalogue from the simplest validation to the most demanding scenario, then the parameter-sensitivity studies and the cost measurements, and closes by returning to the hypothesis criterion by criterion.

5.1 Experimental Protocol

All experiments run through the deterministic headless solver, without rendering or interaction, so that each run is reproducible from a scene definition and a transfer-mode flag alone and any reported timing reflects the simulation only.

The runs, their organisation, and the figures are produced by a single configuration driven orchestration layer rather than by per-experiment scripts. Each scene is defined once as a configuration that fixes its geometry, resolution, timestep, and the set of transfer rules to compare. From that definition the orchestrator launches one deterministic headless run per method, collects the per-stage diagnostics each run exports as a table, and stores the exact command line that produced each run alongside them, so that a run can be reproduced or re-examined without reconstructing how it was launched. A companion viewer reads these tables and plots any recorded quantity against any other, including quantities derived from them such as normalised energies or retention ratios, without rerunning the simulation. Every figure in this chapter is generated through this pipeline from the exported diagnostics, and the tool accompanies the thesis, so that a reader can both reproduce the printed figures and construct views beyond them, including the many secondary diagnostics that are recorded for every run but do not warrant a figure of their own, such as the per-frame CFL number discussed below.

The transfer rule is the single independent variable. Every scene is run for PIC ($\alpha = 0$), FLIP, APIC, and PolyPIC from identical initial states, with all remaining stages, namely forces, extrapolation, boundary enforcement, projection, and advection, left as shared, unmodified code (Section 4.7). The FLIP line in the scene sections is run at the lightly blended setting $\alpha = 0.95$, close to pure FLIP and the common practical default [ZB05; Bri15]. Pure FLIP at $\alpha = 1$ and the full range down to $\alpha = 0$ are examined on their own in the parameter study (Section 5.7).

The timestep is held fixed and adaptive stepping is disabled, so that all methods integrate the same number of identical steps and the step is characterised by a dimensionless CFL number rather than a raw timestep. The baseline step is fixed at $\Delta t = 0.1$, which places the CFL number at order one in the regime that stresses the methods hardest: the mean Courant number of the violent dam break (Section 5.5) is close to unity, between 0.6 and 1.0 across the four transfer rules. Holding the step fixed rather than adapting it to the flow means the realised CFL number is not pinned to one but follows the instantaneous velocity field. It therefore sits an order of magnitude below the target in the quiescent validation scenes, where the characteristic speed is itself of order one and the Taylor-Green and confined vortices (Sections 5.2 and 5.3) run near 0.1, and it rises to and briefly past one in the free-surface scenes, where isolated splash and impact frames reach a small multiple of the target. This spread is a deliberate consequence of fixing the step. The fixed step holds the step count and the integration identical across methods,

which a per-scene or adaptive step would not, and the realised CFL number is recorded for every run so the timestep regime of each measurement is known.

Every scene uses the same spatial sampling, so that the differences between methods reflect the transfer rule rather than the discretisation. The grid spacing is fixed at $\Delta x = 1$ throughout, and each cell is seeded with four particles in two dimensions and eight in three dimensions. The seeding is deterministic but jittered. Particle positions are offset within their cells by a fixed perturbation drawn from a fixed seed, so that the sampling avoids a perfectly regular lattice while every run still reproduces exactly. Grid resolution enters as a variable only in the sensitivity study (Section 5.7), where it is swept on its own, and the grid resolution of an individual scene follows from the domain extent stated in its own section.

Boundaries are free-slip throughout, and no run receives scene- or method-specific tuning, particle reseeding, or added damping, so that any measured difference is attributable to the transfer rule rather than to auxiliary stabilisation (Section 5.10). Every criterion is recorded through the diagnostic defined for it in Section 4.8, so that the same quantity is compared across methods, scenes, and parameter settings.

The harness is calibrated on a uniform-translation case, a body of fluid drifting at constant velocity, which every method must reproduce exactly because the interpolation weights form a partition of unity. There every method held all conserved quantities constant to within $\sim 10^{-14}$, fixing the machine-precision zero point against which the nonzero measurements of the later scenes are read.

5.2 Taylor-Green Vortex

Purpose and setup. This scene measures how strongly the transfer step smooths a rotational field. The Taylor-Green vortex is an exact solution of the incompressible equations, a 2×2 array of counter-rotating vortices whose velocity amplitude decays under viscosity as $\exp(-2\nu k^2 t)$, where k is the wavenumber of the vortex [TG37]. The solver used here is inviscid, so the exact solution is steady and its kinetic energy does not decay. Any energy lost over a run is therefore produced entirely by the discretisation, which makes the scene a clean measurement of numerical dissipation. The array is symmetric, however, so its net angular momentum about the domain centre is zero by construction and cannot separate the methods. The angular-momentum criterion is therefore tested in the confined vortex scene (Section 5.3), and this scene is read solely as the dissipation measurement.

The configuration places the analytic field in a setting the solver can reproduce exactly. A square domain is filled completely with fluid in the all-fluid mode, with no air cells, under zero gravity, and is initialised with a 2×2 array of counter-

rotating vortices,

$$u(x,y) = U_0 \cos(kx)\sin(ky), \qquad v(x,y) = -U_0 \sin(kx)\cos(ky). \tag{5.1}$$

The free-slip walls of the box are placed on the symmetry lines of the array, where the normal velocity of the analytic field is zero, so the boundary reproduces the exact solution rather than imposing a foreign condition. With gravity disabled there is no external forcing, and the field is divergence-free so the pressure projection has almost nothing to remove at the outset. Energy therefore leaves the system only through the discretisation, predominantly through the transfer, so the decay measured over a run is attributable to the transfer rule. The field uses a unit velocity amplitude $U_0 = 1$ and wavenumber $k \approx 0.1$ on a 64×64 domain at the baseline grid spacing $\Delta x = 1$ and seeding of Section 5.1. Each method is run for 180 time units at a fixed timestep as discussed in Section 5.1. The initial state is shown in Figure 5.1.

The recorded series is the normalised kinetic energy $E(t)/E(0)$, defined in Section 4.8 and shown in panel (a) of Figure 5.2 as its decay away from one. Since kinetic energy scales with the square of the velocity amplitude, the energy decays as $\exp(-4\nu_{\text{eff}} k^2 t)$, and a fit of the recorded series returns the effective numerical viscosity

$$\nu_{\text{eff}} = -\frac{1}{4k^2}\,\frac{\mathrm{d}}{\mathrm{d}t}\ln\frac{E(t)}{E(0)}, \tag{5.2}$$

reported per method in panel (b). The fitted value is read alongside the shape of the energy history, since a low ν_{eff} obtained from a noisy or non-monotonic curve is not equivalent to the same value obtained from a clean exponential decay. Secondary to these two readings, the per-stage energy attribution of Section 4.8 confirms that the dissipation accumulates in the two transfer stages rather than in the projection, so that ν_{eff} measures the transfer and not the pressure solve.

Expected behaviour. Because the only decay present is numerical, fitting the measured energy decay to the analytic form recovers an effective numerical viscosity ν_{eff} for each method, with no physical decay to separate out first. The Method chapter predicts that this viscosity orders the methods by how much each transfer smooths the field. PIC, which replaces particle velocities by the cell average every step, is expected to show the largest ν_{eff}. APIC and PolyPIC, which return additional per-particle structure, are expected to show small values, and PolyPIC, which restores the full grid-representable mode set so that the round-trip approaches a lossless projection [FGG+17], is expected to show the smallest. FLIP at $\alpha = 0.95$ is expected to dissipate little as well, only slightly more than APIC and PolyPIC because of the small residual PIC content carried by the blend.

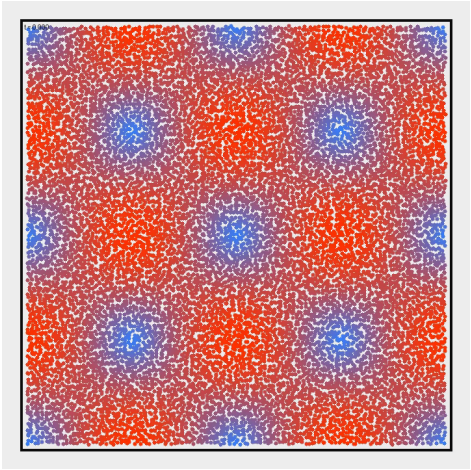


Figure 5.1: Initial state of the Taylor-Green vortex scene. A square free-slip box is filled completely with fluid under zero gravity and seeded with a 2×2 array of counter-rotating vortices (red) following Equation (5.1). The walls lie on the symmetry lines of the array, where the normal velocity of the analytic field vanishes, so the boundaries reproduce the exact solution.

Results. The kinetic energy decays smoothly for all four methods, and a single exponential fits each recorded history with $R^2 \geq 0.99$ (Figure 5.2, panel (a)). Over the 180 time units of the run the normalised kinetic energy falls almost to zero for PIC, to 0.47 for FLIP at $\alpha = 0.95$, to 0.85 for APIC, and to 0.93 for PolyPIC. The fitted effective viscosities follow the same ordering and span more than two orders of magnitude. PIC carries by far the largest value at $\nu_{\text{eff}} = 1.65$, about sixteen times the FLIP figure and almost two orders of magnitude above APIC and PolyPIC. FLIP at $\alpha = 0.95$ follows at 0.102, several times more dissipative than the affine methods, while APIC and PolyPIC sit lowest at 0.022 and 0.010 (Figure 5.2, panel (b)). PolyPIC is therefore about half as dissipative as APIC, a consistent separation, although both values lie so close to the inviscid ideal that the absolute difference between them is small. The per-stage attribution places the bulk of this loss in the two transfer stages. The net transfer removes a cumulative 0.99 of the initial energy for PIC, 0.69 for FLIP, 0.37 for APIC, and 0.29 for PolyPIC, while the projection removes a far smaller amount, between 0.002 and 0.02, over the same run.

Discussion. The measured dissipation orders the methods as the Method chapter predicts. PIC is the most dissipative by a wide margin, APIC and PolyPIC are the least dissipative, and FLIP at $\alpha = 0.95$ falls between them. APIC and PolyPIC are close but not identical. PolyPIC is the least dissipative method, at about half the effective viscosity of APIC. The separation is consistent rather than dramatic, since

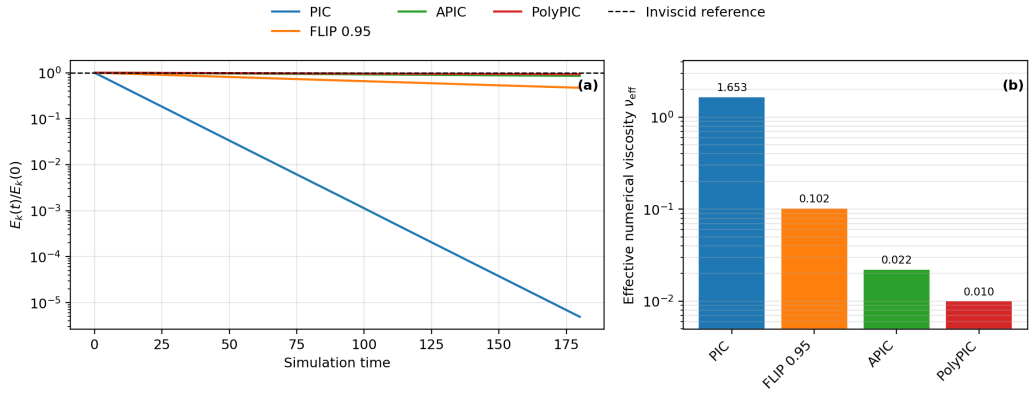


Figure 5.2: Dissipation of the Taylor-Green vortex for the four transfer methods. (a) Normalised kinetic energy $E(t)/E(0)$ against simulation time, with the inviscid analytic solution as the flat reference at one. (b) Effective numerical viscosity ν_{eff} fitted per method from the energy decay. PIC dissipates most strongly, APIC and PolyPIC least, and FLIP at $\alpha = 0.95$ lies just above them.

both methods sit so near the inviscid ideal that the absolute gap between them is small. It is real nonetheless, and shows that the higher PolyPIC modes recover a little energy that the affine state alone does not, even on this smooth rotational field. FLIP at $\alpha = 0.95$ decays as a clean exponential but dissipates several times more than APIC, not marginally more. The per-stage attribution supports reading ν_{eff} as a property of the transfer, since the transfer stages carry most of the energy loss for every method while the projection contributes a small amount throughout.

5.3 Confined Vortex

Purpose and setup. This scene is the chapter's primary test of the angular-momentum criterion. It places a single compact, single-signed vortex at the centre of the domain, a flow that carries a large, well-defined net angular momentum, and measures how faithfully each transfer rule preserves that rotation over time. It supplies a rotational test that the Taylor-Green vortex (Section 5.2) cannot, since the symmetric Taylor-Green array has zero net angular momentum by construction whereas a confined swirl has a large one.

The configuration is a square domain filled completely with fluid in the all-fluid mode, with no air cells, under zero gravity and free-slip walls, seeded with a single swirl defined by the stream function

$$\psi(r) = A\left(1 - \frac{r^2}{R^2}\right)^2 \quad (r \le R), \qquad \psi(r) = 0 \quad (r > R), \tag{5.3}$$

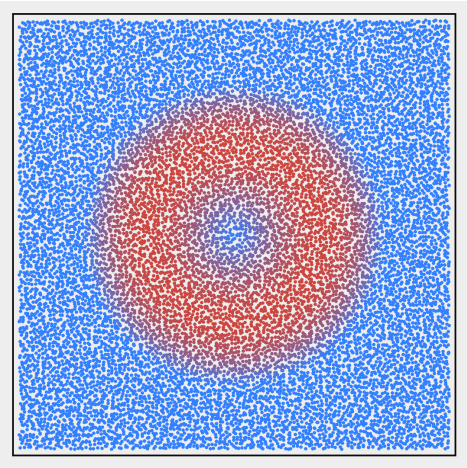


Figure 5.3: Initial state of the confined vortex scene. A single single-signed swirl of radius $R = 0.7$ times the domain half-extent is seeded at the centre of a square box filled completely with fluid under zero gravity, following Equation (5.3), with the surrounding fluid at rest.

with velocity $\mathbf{u} = \nabla^{\perp}\psi = (\partial_y\psi, -\partial_x\psi)$ and support radius $R = 0.7$ times the domain half-extent, about 21.7 cells on the 64×64 grid. The field is divergence-free, single-signed, and a steady solution of the inviscid equations, so without numerical error the core would rotate indefinitely without spreading. The fluid outside R is at rest, which keeps the swirl confined while leaving it embedded in a filled box rather than surrounded by void. The domain, grid spacing, and seeding follow the baseline of Section 5.1, and each method is run for 120 time units at a fixed timestep.

The experiment serves the same diagnostic purpose as the rotating-circle comparison of Jiang et al., but adapts the configuration to the available boundary treatment [JSS+15]. Instead of an isolated circular particle region, the compact vortex is embedded in a square domain filled with otherwise stationary fluid.

The headline reading is the core-concentrated angular momentum. Comparing it with the total orbital angular momentum reveals what happens to rotation that leaves the initial core. If the core quantity decreases while the total remains close to its initial value, the rotation has primarily spread into the surrounding fluid. A simultaneous decrease in the total indicates that redistribution alone cannot explain the loss. The core quantity, $L_z^{\mathrm{core}}(t)$, is the orbital angular momentum of the particles within radius R of the centre of mass (Equation (4.23)) and is shown in Figure 5.4. Peak particle speed and kinetic energy, each relative to their initial values, provide additional measures of how well the concentrated swirl survives.

Expected behaviour. The expectations follow from the Method chapter. APIC carries the local velocity gradient in its affine state, whose antisymmetric part represents local rotation, so it should transport the swirl as a coherent rotating core and hold L_z^{core} near its initial value [JSS+15; JST17]. PolyPIC adds the remaining grid-representable modes and should preserve the core at least as well [FGG+17]. PIC replaces each particle velocity with an interpolated grid velocity after every step, which repeatedly removes sub-cell rotation. It is therefore expected to diffuse the swirl outward and lose both core angular momentum and kinetic energy. FLIP at $\alpha = 0.95$ retains more particle velocity history but carries no exact angular-momentum guarantee, so it should fall between the affine methods and PIC. The predicted order in core retention is PolyPIC, then APIC, then FLIP, with all three above PIC. The total is expected to decay less than the core because it also counts redistributed rotation.

Results. The final core angular-momentum retention separates the methods into three levels (Figure 5.4). After 120 time units, PIC retains 0.12 of its initial core angular momentum. FLIP retains 0.79, while APIC and PolyPIC retain 0.95 and 0.97, respectively. The predicted order is maintained throughout the final comparison. PolyPIC finishes slightly above APIC, and both remain distinctly above FLIP.

The total orbital angular momentum is retained more strongly than the core quantity, but it also distinguishes the methods. PIC retains 0.50 of its initial total, compared with 0.98 for FLIP, 0.97 for APIC, and 0.99 for PolyPIC. Peak-speed retention follows the same ordering at 0.11, 0.74, 1.01, and 1.03. The corresponding kinetic-energy retentions are 0.03, 0.57, 0.90, and 0.94.

Discussion. The core measurement confirms the predicted ordering and replaces the previous grouping of the three less dissipative methods. FLIP preserves substantially more concentrated rotation than PIC, but it remains clearly below APIC and PolyPIC. PolyPIC exceeds APIC by approximately two percentage points in core retention. The difference is small in absolute terms, but it is consistent with the total-angular-momentum, peak-speed, and kinetic-energy readings. The peak speeds of APIC and PolyPIC rise slightly above their initial values even though their total kinetic energies decrease, which indicates local speed amplification rather than net energy growth.

The difference between core and total retention distinguishes redistribution from loss of the measured total. For PIC, the increase from 0.12 core retention to 0.50 total retention shows that part of the initial rotation spreads beyond the core. The fall of the total to one half also shows that redistribution alone cannot account for the result. FLIP retains 0.79 in the core and 0.98 in total, so its loss of

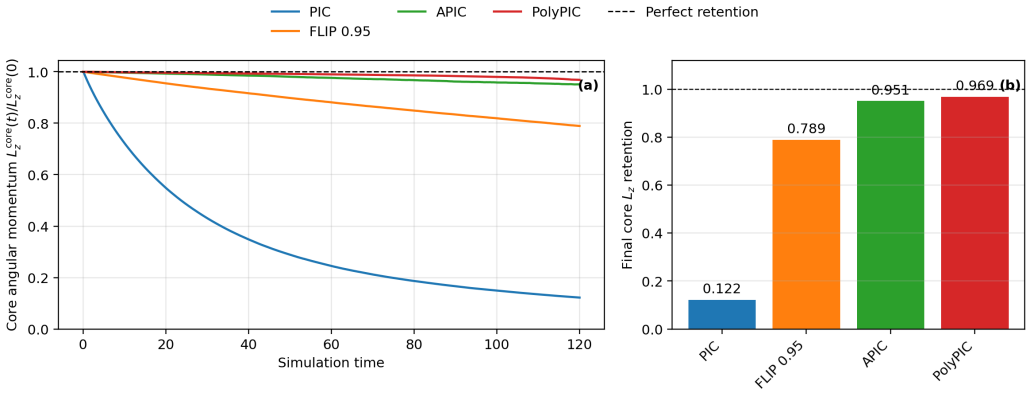


Figure 5.4: Core-concentrated angular momentum of the confined vortex for the four transfer methods. (a) Core angular momentum $L_z^{\text{core}}(t)/L_z^{\text{core}}(0)$ against simulation time, with perfect retention as the flat reference at one. (b) Final core retention per method. PIC loses most of the concentrated rotation, FLIP retains an intermediate amount, and APIC and PolyPIC remain close to their initial values.

concentration is primarily redistribution. The small differences between the core and total readings for APIC and PolyPIC show that both methods preserve not only the measured orbital angular momentum but also its concentration within the original support region.

These diagnostics contain the particle-centre orbital contribution rather than the internal affine contribution carried by APIC and PolyPIC. They therefore measure preservation of the visible bulk rotation, not the complete theoretical angular momentum of the enriched particle state.

5.4 Settling Pool

Purpose and setup. This scene isolates ringing in the particle velocity field, a primary simulation-quality criterion of this thesis. A rectangular plug falls into a pool under gravity and generates a transient containing impact, surface motion, and pressure-driven flow. Hydrostatic rest provides the equilibrium reference, but grid-resolved motion may persist while the pool approaches it. Brackbill identifies ringing as a low-speed PIC instability caused by aliasing between the particle and grid representations [Bra88]. The settling phase therefore provides a suitable setting for comparing how strongly each transfer method retains motion that the grid cannot represent. Ringing is identified through this unresolved particle motion rather than through kinetic energy alone.

The pool initially occupies the lower 38% of the 64×64 domain. A centred plug spans 14% of the domain width and extends vertically from 56% to 72% of

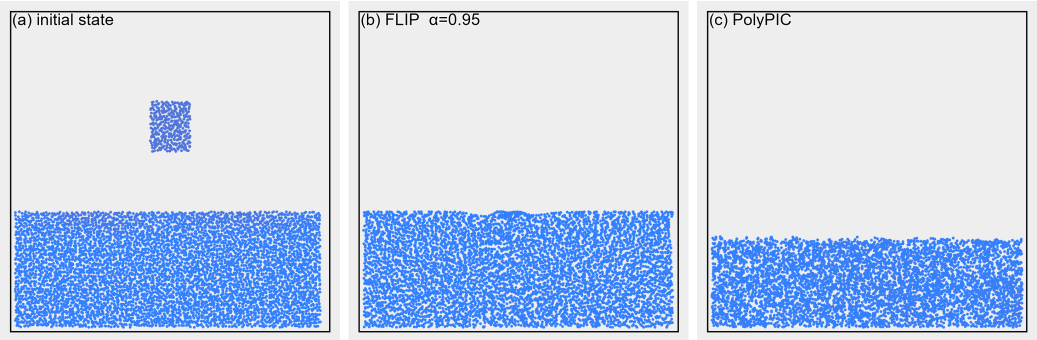


Figure 5.5: Settling pool. (a) Initial state, a rectangular fluid plug falling towards a perturbed pool under gravity in a square domain with free-slip walls. (b, c) Particle distribution at $t = 180$ for FLIP with $\alpha = 0.95$ and PolyPIC at the refined timestep $\Delta t = 0.025$. FLIP develops uneven clumping and voids, whereas PolyPIC stays evenly sampled. Panels (b) and (c) are a qualitative illustration, and the precise pattern depends on the scene and the particle seeding.

the domain height. It begins with a downward velocity of 2.5. The upper 10% of the pool receives a sinusoidal surface perturbation with horizontal and vertical amplitudes of up to 2.0 and 0.75, respectively. The domain uses free-slip walls, gravity, the baseline spacing $\Delta x = 1$, and a fixed timestep $\Delta t = 0.1$, the same as the previous experiments. Each run contains 5932 particles and continues for 180 time units.

The comparison includes PIC, FLIP with $\alpha = 0.95$, 0.99, and 1.00, APIC, and PolyPIC. All runs use the same deterministic initial state. Figure 5.5 shows the initial pool and falling plug.

The primary reading is the scene-normalised residual energy e^n of Equation (4.32). Its geometric mean over the fixed late-time window $120 < t \leq 180$ gives the ringing score $\bar{e}$ of Equation (4.33). The companion fraction f^n_{res} of Equation (4.31) measures how much of each method's current particle kinetic energy cannot be reconstructed on the grid. Reading both quantities separates unresolved particle motion from legitimate energy retained by the simulation.

Expected behaviour. PIC overwrites the particle velocities with a grid reconstruction after every step. It should therefore drive both the residual energy and the physical transient rapidly towards zero. A low ringing score for PIC is expected to reflect strong damping rather than faithful retention of motion. FLIP preserves particle velocity history that is weakly represented on the grid. Its residual should persist at late times and increase as α approaches the pure FLIP update. APIC and PolyPIC reconstruct their enriched particle states from the grid without retaining a

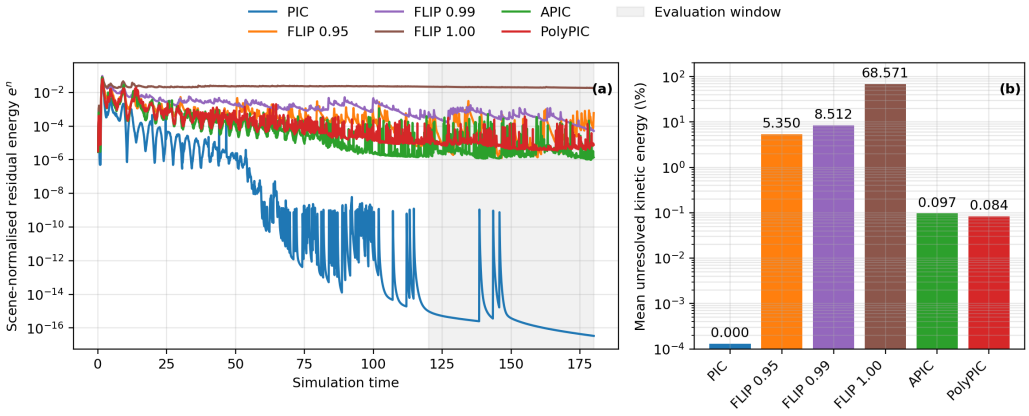


Figure 5.6: Ringing diagnostics for the settling pool. (a) Scene-normalised residual energy e^n on a logarithmic scale. The shaded region marks the fixed evaluation window $120 < t \leq 180$ used for the ringing score $\bar{e}$. (b) Mean particle-relative unresolved kinetic-energy fraction over this window. The PIC value is below 0.001% and rounds to zero at the displayed precision.

separate FLIP-style velocity history. They should therefore keep the unresolved fraction low while preserving more grid-resolved motion than PIC.

Results. The late-time ringing scores separate the three FLIP blends from PIC, APIC, and PolyPIC (Figure 5.6). PIC reaches 1.08×10^{-12}, the smallest value in the comparison. APIC and PolyPIC record 3.67×10^{-6} and 8.73×10^{-6}, respectively. The score rises to 2.69×10^{-5} for FLIP at $\alpha = 0.95$, 3.33×10^{-4} at $\alpha = 0.99$, and 2.01×10^{-2} for pure FLIP. The $\alpha = 0.95$ score is approximately 7.3 times the APIC score and 3.1 times the PolyPIC score. Pure FLIP exceeds the $\alpha = 0.95$ blend by approximately a factor of 745.

The particle-relative reading preserves the same broad separation. Over the evaluation window, the mean unresolved fractions are 5.35%, 8.51%, and 68.6% for the three increasing FLIP values. The corresponding means are 0.097% for APIC and 0.084% for PolyPIC. Their mean absolute residual energies are almost equal at 0.617 and 0.610, while PolyPIC retains a larger mean particle kinetic energy of 582.8 compared with 382.4 for APIC. The PIC fraction falls to approximately 0.00013% as its particle motion is damped away.

Discussion. The result confirms that incremental particle velocity memory is the dominant source of ringing in this scene. The residual increases sharply as the FLIP blend approaches one. Pure FLIP maintains a persistent plateau in both readings, with more than two thirds of its late-time particle kinetic energy absent from the

reconstructed grid field. A five-percent PIC contribution reduces this instability substantially, but the $\alpha = 0.95$ blend still remains above APIC and PolyPIC.

The near-zero PIC score does not identify it as the most faithful method. Its particle motion and unresolved residual vanish together under repeated grid reconstruction. Read alongside the strong PIC dissipation measured in the Taylor-Green vortex (Section 5.2), this is over-damping rather than stable preservation of the transient.

PolyPIC retains more kinetic energy than APIC, while their absolute residual energies are almost equal. A smaller fraction of PolyPIC's remaining motion is therefore unresolved by the grid. On this relative measure, PolyPIC is slightly more stable against ringing in this scene, although APIC has the lower absolute ringing score.

The difference is consistent with how the methods handle particle degrees of freedom. FLIP retains velocity components that the grid cannot represent, which allows noise to persist. APIC and PolyPIC instead reconstruct their particle states from the grid after every step. PolyPIC also stores the grid-representable cross-mode, so its additional state preserves resolved motion rather than particle-only noise (Sections 4.2 and 4.4). This retained noise is also visible in the late-time particle distribution, where FLIP develops uneven clumping and voids while PolyPIC stays evenly sampled (Figure 5.5, panels (b) and (c)).

The timestep dependence of this separation is examined in Section 5.7. The same residual criterion is revisited in the dam break (Section 5.5) to test whether the ordering survives a violent boundary-dominated flow.

5.5 Dam Break

Purpose and setup. The preceding scenes each isolate one aspect of transfer behaviour. The dam break combines them. A fluid column collapses under gravity, spreads across the domain, and repeatedly collides with the walls, so dissipation, ringing, free-surface motion, and boundary interaction all act at once. Jiang et al. use a comparable dam-break scene to demonstrate APIC under complex free-surface motion [JSS+15]. The scene therefore tests whether the orderings established in the controlled experiments survive when these effects occur together.

The fluid begins at rest as a column in a free-slip box under gravity. At the baseline grid spacing, the seeded region occupies the left third of the domain, beginning one cell from the lower and left walls and ending two cells below the upper wall. The remaining resolution and seeding settings follow Section 5.1. The initial state is shown in Figure 5.7.

The stability reading reuses the scene-normalised residual energy and particle-relative unresolved fraction from Section 5.4. The same fixed window $120 < t \leq$

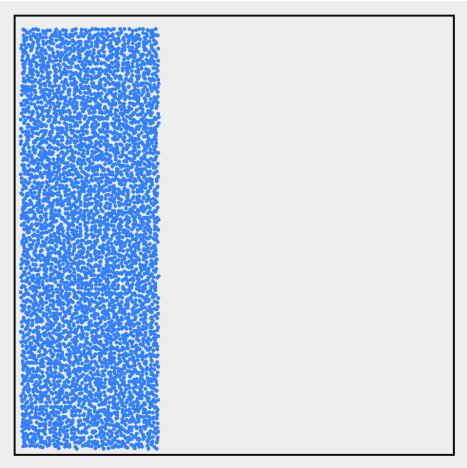


Figure 5.7: Initial state of the dam break scene. A column of fluid is released from rest in a free-slip box under gravity and collapses across the domain against the walls, at the baseline resolution and seeding of Section 5.1.

180 is used for the ringing score and the mean unresolved fraction. Particle kinetic energy and the binary fluid-cell measure provide companion readings of retained motion and free-surface sampling.

Expected behaviour. Because the dam break combines effects that the previous scenes isolated, the per-method trends established there are expected to recur. Their recurrence would show that the orderings are properties of the transfer rules rather than artifacts of the idealised single-effect scenes. FLIP should continue to ring after the strongest motion has passed. PIC should damp the small-scale motion of the splash, while APIC and PolyPIC should preserve more of it without the same ringing.

Results. The initial collapse produces comparable kinetic-energy peaks for all methods, after which their histories separate (Figure 5.8, panel (a)). Over the evaluation window, PolyPIC retains the largest mean particle kinetic energy at 2861.8. The corresponding means are 2140.6 for FLIP with $\alpha = 0.95$, 1526.3 for APIC, and 263.8 for PIC. The dam break is also the most violent of the baseline scenes. Its peak realised CFL number ranges from 5.23 to 6.07 across the four methods.

The residual readings distinguish retained grid-resolved motion from particle motion that the grid does not reproduce (Figure 5.8, panels (b) and (c)). The ringing scores are 2.31×10^{-8} for PIC, 8.63×10^{-6} for FLIP, 7.90×10^{-7} for APIC, and 3.31×10^{-6} for PolyPIC. Their mean unresolved fractions over the same

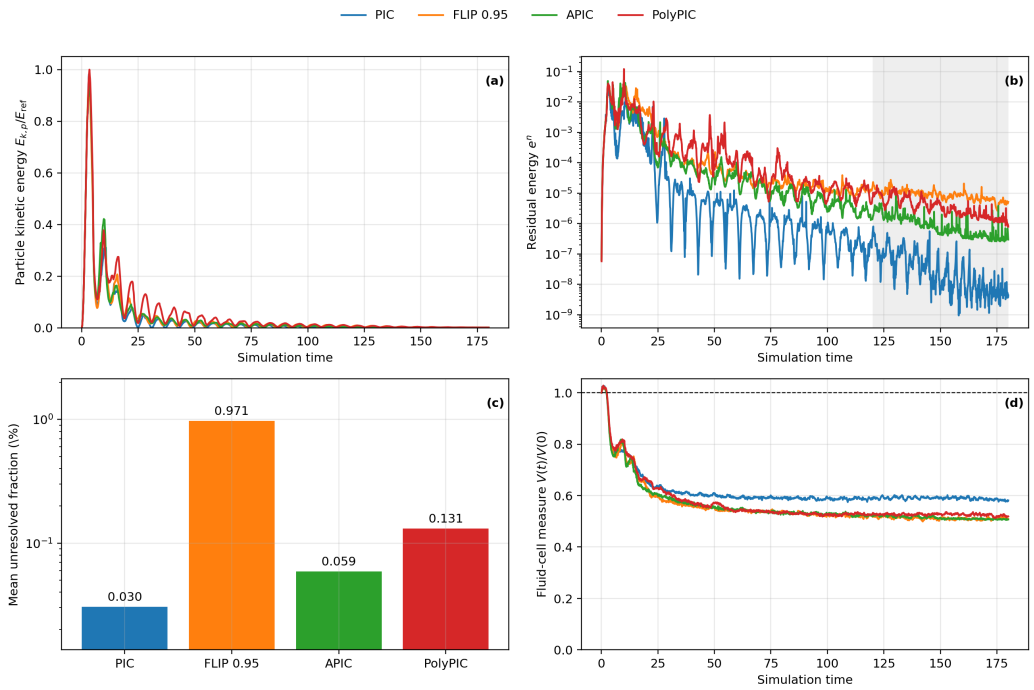


Figure 5.8: Integrated dam-break diagnostics. (a) Particle kinetic energy relative to the shared scene reference energy. (b) Scene-normalised residual energy on a logarithmic scale, with the evaluation window $120 < t \leq 180$ shaded. (c) Mean particle-relative unresolved kinetic-energy fraction over this window on a logarithmic scale. (d) Binary fluid-cell measure relative to its initial value.

window are 0.030%, 0.971%, 0.059%, and 0.131%, respectively. APIC therefore has the lowest absolute and relative residual among the three methods that retain substantial late motion. PolyPIC retains the most kinetic energy, but its larger unresolved fraction shows that its higher residual cannot be attributed to the energy scale alone.

The binary fluid-cell measure decreases for every method during the collapse (Figure 5.8, panel (d)). At the final sample, PIC retains 0.581 of its initial measure. The values for FLIP, APIC, and PolyPIC are 0.508, 0.507, and 0.519, respectively. All runs retain their initial 4880 particles and report no invalid or out-of-bounds particles. Their late mean post-projection RMS divergence remains between 2.5×10^{-9} and 3.0×10^{-9}.

Discussion. The integrated scene preserves the main distinction between damping and ringing found in the controlled experiments. PIC produces the smallest residual because repeated grid reconstruction removes both unresolved motion and the physical transient. Its low score therefore reflects the same strong dissipation

observed in the Taylor-Green vortex, rather than faithful preservation of a clean splash (Section 5.2). FLIP retains much more motion, but its residual energy and unresolved fraction are the largest of the four methods. The five-percent PIC blend is not sufficient to match the ringing stability of either enriched transfer.

The comparison between APIC and PolyPIC differs from the settling pool. PolyPIC retains more kinetic energy in the dam break, but APIC has both the lower ringing score and the lower unresolved fraction. APIC is therefore more stable against ringing in this scene. PolyPIC still remains substantially below FLIP in both residual readings while retaining the most late motion. Together with the settling-pool result, this shows that the stability ordering between the two enriched transfers depends on the flow rather than following directly from the number of represented particle modes.

The decrease in the fluid-cell measure does not indicate comparable material loss. The particle count remains constant, no particles become invalid or leave the domain, and the projected grid field remains divergence-free to numerical precision. The occupied-cell estimate is instead sensitive to how particles redistribute within the domain and how the free surface crosses the binary fluid-cell classification. It therefore exposes a free-surface sampling limitation of this diagnostic, but not a failure of the pressure projection. The comparatively higher estimate for PIC is consistent with its stronger numerical damping. Since repeated PIC transfers reduce kinetic energy and particle velocities, particles are less likely to separate from the bulk fluid or form thin structures that are poorly captured by the occupied-cell count. A higher estimated volume for PIC therefore does not imply a more accurate flow as in this case it reflects the stabilising effect of numerical dissipation.

The dam break consequently confirms the broad cross-scene trade-off under a violent boundary-dominated flow. PIC is stable through damping, FLIP preserves motion with the largest unresolved component, and the enriched transfers keep most of their retained motion representable on the grid. The difference between APIC and PolyPIC is smaller and scene-dependent, with PolyPIC favouring retained motion here and APIC favouring ringing stability.

5.6 Compressed Hyperelastic Square

Purpose and setup. The preceding scenes evaluate the transfer methods inside the incompressible-fluid solver. This scene moves the same comparison into a deformable-solid setting, where a hyperelastic body repeatedly exchanges kinetic and elastic energy as it oscillates. Transfer dissipation then accumulates across many cycles rather than appearing only during a single fluid transient, and the released deformation drives spatially varying velocity gradients instead of the near-uniform motion of a settling or collapsing fluid. Fu et al. use a compressed elastic

body of this kind to demonstrate PolyPIC in the material point method [FGG+17]. Repeating it here places an established deformable-material benchmark inside the same implementation, diagnostics, and method comparison used throughout this evaluation, so the transfer behaviour observed in the fluid scenes can be checked against a recognised elastic test under identical measurement and further analyzed.

The body is realised in the collocated quadratic B-spline MPM setting of Section 4.6, whose nine-mode representable space sets the basis available to each transfer rule. The computational domain is a 64×64 grid with spacing $\Delta x = 1$. A square body covering 12×12 cells is centred in the domain. Each occupied cell contains a deterministic 2×2 particle arrangement, giving 576 particles in total. Every particle has mass $1/4$, so each occupied cell initially carries unit mass. The particle layout is shown in Figure 5.9.

The body begins at rest with the uniform elastic deformation gradient

$$\mathbf{F}^{0}_{E,p} = 0.9\,\mathbf{I}. \tag{5.4}$$

This state stores compressive elastic energy throughout the body while leaving the particles on a regular lattice, so the stored strain rather than the displayed positions drives the subsequent motion. Gravity is disabled, so releasing the initial strain is the only source of motion. All methods use the same particles, material parameters, grid, interpolation kernel, and fixed timestep $\Delta t = 10^{-5}$. Each run advances one million steps over 10 simulated time units. The compared configurations are PIC, blended FLIP with $\alpha = 0.95$, APIC, and PolyPIC carrying the full nine-mode representable space, each starting from the same particle state.

The primary reading is the normalised total mechanical energy, formed from grid kinetic energy and particle elastic energy. Its minimum and final values quantify the damping accumulated over the oscillation, and the kinetic and elastic parts are read separately to show their exchange. The body width and height track the geometric response, while the maximum particle speed and the finiteness of the particle state expose instability or failure. The amplitude of the beyond-affine mode coefficients, that is the cross-mode and quadratic-mode content APIC cannot represent, is recorded as a supporting diagnostic of whether the motion excites particle state outside the affine subspace. It is read as a description of the represented motion rather than as a quality score.

Expected behaviour. The stored elastic energy is expected to drive the body to expand, overshoot its undeformed state, and oscillate, with kinetic and elastic energy exchanging while the total remains constant in the absence of numerical loss. The compressed square is well suited to provoking ringing because the repeated expansion and rebound inject strong grid-scale velocity variation many times over, while the near-rest phases between rebounds expose any particle-scale motion

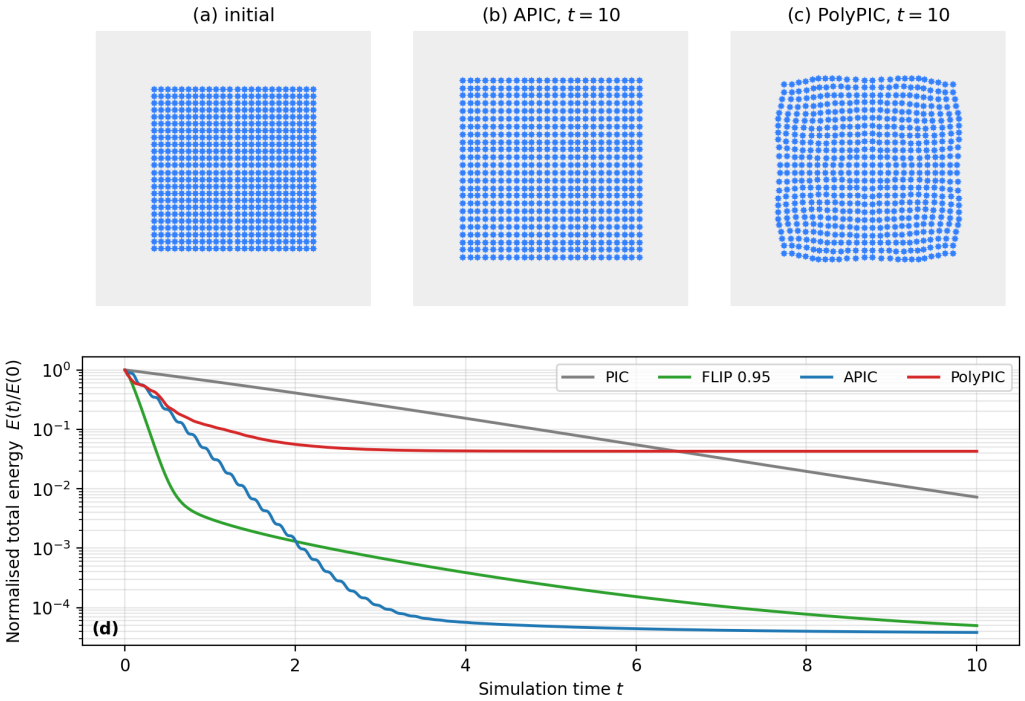


Figure 5.9: Compressed hyperelastic square, with the body region cropped from the full 64×64 MPM domain. (a) Initial particle layout: the 576 particles form a centred 24×24 lattice over 12×12 grid cells, the compression being stored in the deformation gradient of Equation (5.4) rather than in the displayed positions, and the body is released from rest without gravity. (b, c) Particle configuration at $t = 10$ for APIC and PolyPIC: APIC relaxes to a regular, stress-free square, whereas PolyPIC freezes a wider, non-uniformly curved shape that still holds elastic energy. PIC and the FLIP blend, not shown, remain close to the initial shape after barely releasing the strain. (d) Normalised total mechanical energy against time on a logarithmic scale for the four methods.

that the coherent breathing oscillation does not explain. Ringing would appear here as irregular, grid-unresolved particle velocity superimposed on the smooth deformation, the same unresolved-motion signature identified in the settling pool (Section 5.4), rather than as a clean decay of the oscillation.

Following the orderings established in the fluid scenes, PIC is expected to damp the oscillation most rapidly, because repeated grid reconstruction removes local velocity variation. FLIP is expected to retain more of the motion but may carry particle-scale noise, the elastic-setting counterpart of the ringing it shows in the fluid scenes. APIC and PolyPIC carry richer local velocity representations that should preserve the coherent deformation with less damping and without the same noise, and PolyPIC additionally retains the beyond-affine grid-representable

modes that the released deformation excites. Whether this additional state yields a measurable advantage over APIC in this scene is left as the hypothesis the completed runs are expected to test, rather than assumed.

Results. The stored strain converts into motion to very different degrees. The peak kinetic energy reaches 85.0% of the initial energy for APIC and 60.0% for PolyPIC, against only 2.88% for the FLIP blend and 0.009% for PIC. The two enriched transfers therefore drive a genuine oscillation, while PIC stays almost stationary and the blend barely moves. Total mechanical energy decays for every method, but this alone does not measure retained motion, because suppressed strain remains stored as elastic energy. At $t = 0.5$, PIC still holds 80.7% of the initial energy, almost all of it elastic, in a body that has scarcely moved.

By the final sample the methods separate into two outcomes (Figure 5.9d). PIC, the FLIP blend, and APIC have dissipated nearly all energy, to 0.722%, 0.005%, and 0.004% of the initial value. APIC overshoots to a width of 13.758 and then relaxes to 12.781, recovering the stress-free width of 12.78 almost exactly and returning to a regular square (Figure 5.9b). PolyPIC instead retains 4.272% of the initial energy, stored almost entirely as elastic strain at 422.4 against APIC's 0.378, and comes to rest at a width of 13.095 in a visibly curved shape, wider than the stress-free state (Figure 5.9c). Its residual energy is held in a deformed configuration rather than in continuing motion.

The release excites the PolyPIC modes outside the affine space, with peak RMS coefficients of 0.844 for the bilinear and 1.143 for the quadratic modes, which decay as the body slows. The other methods carry none. The ringing signature then appears in a single method. The FLIP blend is alone in retaining notable unresolved, grid-irreproducible particle motion, its unresolved kinetic-energy fraction rising to about 50% during the run and remaining near 29% at late times, while PIC, APIC, and PolyPIC stay at the numerical floor. The absolute energy held in this motion is nonetheless negligible, falling to a late residual of 7.04×10^{-6}, because the heavily damped run retains almost no motion for it to occupy. PolyPIC's retained energy, by contrast, sits at the residual floor and is therefore coherent grid-resolved deformation rather than noise. All runs keep their 576 particles with positive deformation determinants and report no invalid or out-of-bounds states.

Discussion. The near-zero kinetic peaks of PIC and the FLIP blend are the deformable-solid counterpart of the strong PIC damping seen in the fluid scenes. Both relax the body through slow expansion rather than oscillation, because each transfer removes the velocity the elastic forces generate. The blend makes the timestep effect from the parameter study (Section 5.7) concrete: its five-percent PIC fraction is applied at every one of the million steps the fine timestep requires,

so its accumulated dissipation approaches PIC's and it releases only 2.88% of the stored energy. The blend nonetheless reproduces the predicted ringing ordering. It is the only method to retain unresolved particle motion, the same signature FLIP shows in the fluid scenes, while the enriched transfers reconstruct cleanly and stay at the floor. That ringing is present as a fraction but not as a magnitude, because the dissipation that suppresses the release also leaves almost no motion for it to occupy. The compressed square therefore confirms which method rings without provoking strong ringing at this timestep.

APIC and PolyPIC both turn the stored strain into a real oscillation, the elastic analogue of the detail they preserve in the fluid scenes, and differ in their late state. APIC dissipates the oscillation but relaxes to the correct stress-free shape, whereas PolyPIC dissipates less and retains a residual, non-uniform deformation that still holds elastic energy, consistent with the quadratic modes it alone excites. Fu et al. report the same broad tendency for the additional PolyPIC modes to reduce transfer damping in a compressed elastic body [FGG+17]. The retained energy is genuine and grid-resolved rather than noise, but here it sits in a strained shape after the motion has nearly stopped, so it is not in itself a better outcome than APIC's clean return to equilibrium. The PolyPIC modes also carry a small non-physical regularisation, so the magnitude of this residual deformation is a qualitative tendency rather than an exact figure.

The scene therefore carries the fluid ordering into a deformable solid. PIC and the heavily stepped FLIP blend suppress the motion, while the enriched transfers release it. Between the two enriched transfers the scene does not crown a single winner, mirroring the scene-dependent relationship between APIC and PolyPIC found in the settling pool and dam break (Sections 5.4 and 5.5): APIC recovers the equilibrium configuration most faithfully, while PolyPIC preserves the most energy, at the cost of a residual deformation the affine transfer does not produce.

5.7 Parameter Sensitivity

Purpose and setup. The scene sections hold the numerical parameters fixed and vary only the transfer rule. This section tests whether their method orderings survive changes in blend, timestep, and resolution. The settling pool (Section 5.4) provides both the primary ringing criterion and the retained-energy reading in one scene. Each sweep records the ringing score $\bar{e}$ of Equation (4.33) and the retained particle kinetic energy, and is anchored on the baseline configuration of Section 5.1, $\Delta t = 0.1$ and $\Delta x = 1$.

Three parameters are varied. The PIC/FLIP blend α is swept from the strongly damped low-blend regime to pure FLIP, the timestep is varied around the baseline up to the point where the late-time Courant number exceeds one, and the grid

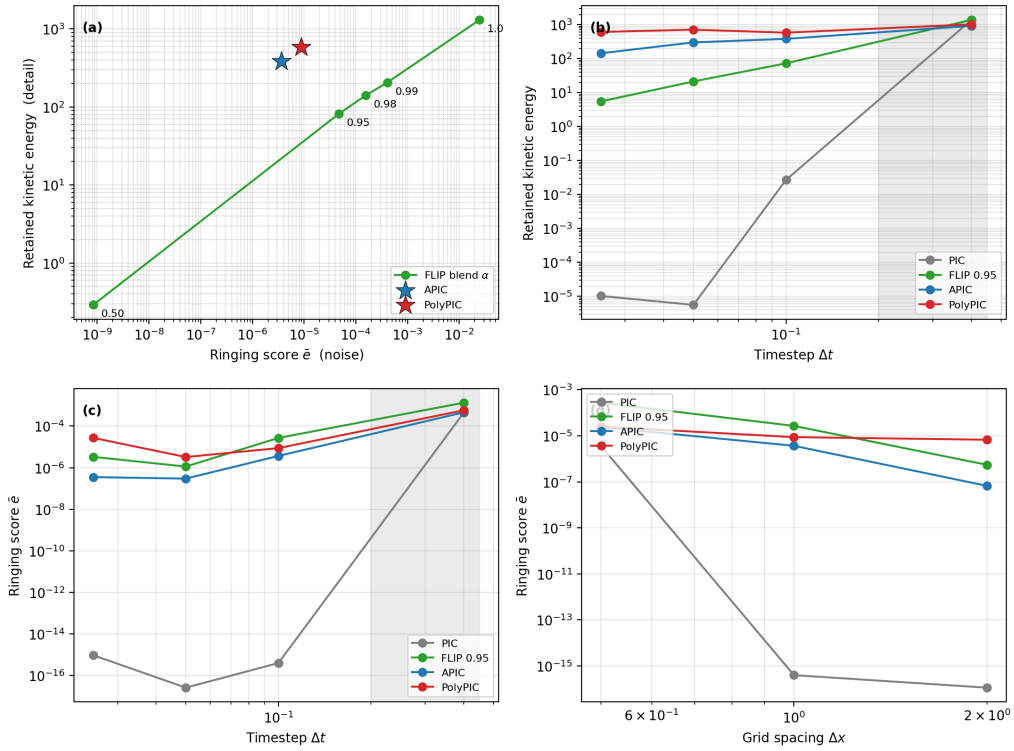


Figure 5.10: Parameter sensitivity on the settling pool, all quantities read over the late-time window $120 < t \leq 180$ on logarithmic axes. (a) Blend trade-off: retained kinetic energy against the ringing score $\bar{e}$ as the PIC/FLIP blend α is swept from 0.5 to 1 (green, with α annotated), with APIC and PolyPIC marked as parameter-free references. (b) Retained kinetic energy and (c) ringing score against the timestep Δt for the four methods; the shaded band marks steps whose late-time Courant number exceeds one. (d) Ringing score against the grid spacing Δx.

spacing is refined and coarsened by a factor of two. APIC and PolyPIC carry no blend parameter and enter as fixed references.

Expected behaviour. The blend is expected to trade its two effects against each other, so that raising α recovers detail but also admits ringing, with no single value optimal across the trade-off. APIC and PolyPIC are expected to reach the favourable region of that trade-off without a parameter to set. The fixed-timestep resolution sweep tests whether the method ordering survives refinement, while a complementary check at constant $\Delta t/\Delta x$ examines how the result changes when the timestep scales with the cell size. Neither experiment is treated as a formal spatial-convergence study.

Results. The blend traces a single trade-off curve (Figure 5.10a). As α rises from 0.95 to pure FLIP, the retained energy grows from 82 to 1307 while the ringing score climbs from 4.7×10^{-5} to 2.5×10^{-2}, a far steeper rise that accelerates as $\alpha \to 1$. APIC and PolyPIC lie off this curve. They retain 382 and 583 at ringing scores of 3.7×10^{-6} and 8.7×10^{-6}, more retained energy than the $\alpha = 0.95$ blend at more than an order of magnitude less ringing, and the blend reaches their retained energy only as $\alpha \to 1$, where its ringing is some three orders of magnitude higher. The lightly blended $\alpha = 0.95$ used in the scenes sits near the knee of the curve, consistent with the common practical default [ZB05; Bri15].

Refining the timestep separates the methods by how much grid-representable motion their transfer preserves (Figure 5.10b). As Δt falls from 0.1 to 0.025, the retained energy of the $\alpha = 0.95$ blend drops from 73 to 5.5, while PolyPIC holds between 583 and 714 and APIC falls more gently from 382 to 144. PIC retains almost no motion at any step. The ringing score stays low across this range for every method (Figure 5.10c) and rises sharply only at $\Delta t = 0.4$, where the late-time Courant number passes one and the score of all four methods, including PIC, jumps to between 4×10^{-4} and 1.3×10^{-3} as the settled state ceases to be stable.

Refining the grid raises the ringing score rather than reducing it (Figure 5.10d). Halving Δx increases the FLIP score from 2.7×10^{-5} to 2.7×10^{-4} and the APIC score from 3.7×10^{-6} to 2.4×10^{-5}, and the method ordering is preserved at every resolution, with FLIP highest and APIC and PolyPIC below it. Because the timestep is held fixed, the finer grid also runs at a higher Courant number, from about 0.15 at $\Delta x = 2$ to near one at $\Delta x = 0.5$, which contributes to the increase.

To separate grid refinement from the rising Courant number in the fixed-timestep sweep, a complementary settling-pool check couples

$$(\Delta x, \Delta t) = (2, 0.20), (1, 0.10), (0.5, 0.05),$$

keeping $\Delta t / \Delta x = 0.1$. The study is only nominally CFL-matched. For the methods retaining appreciable motion, the mean realised CFL over $120 < t \leq 180$ rises from 0.096–0.153 on the coarsest grid to 0.236–0.277 on the finest. The broad ordering persists: PIC remains at the residual floor, FLIP has the largest ringing score, and the enriched transfers lie between them, although APIC and PolyPIC exchange order at the finest level. The scores are not monotonic. From coarse to fine, FLIP records 1.15×10^{-4}, 2.77×10^{-5}, and 5.97×10^{-5}. At the finest level, the unresolved fractions are 0.33%, 0.027%, and 0.039% for FLIP, APIC, and PolyPIC. Particle counts remain constant, no invalid or out-of-bounds states occur, and every pressure solve converges. The weaker and non-monotonic response relative to the fixed-timestep sweep shows that the original increase was not a pure resolution effect and is consistent with an influence of the changing

timestep-to-grid ratio. Because the coupled check also changes the number of transfer cycles and does not produce exactly equal realised CFL numbers, neither sweep isolates a single cause or establishes spatial convergence.

Discussion. The FLIP blend is the most parameter-sensitive method. Its retained energy varies more than tenfold with the timestep, and its ringing varies by three orders of magnitude with α. Smaller steps apply the damping from its PIC fraction more often. PolyPIC remains almost insensitive to the timestep, while APIC loses more motion as the number of transfers grows. This difference is consistent with PolyPIC retaining the cross-mode that APIC drops.

The fixed-timestep refinement sweep does not separate resolution from the rise in CFL. The coupled check weakens the trend but preserves the broad separation between FLIP and the enriched transfers. It supports this method grouping, but neither a monotonic resolution law nor spatial convergence. The blend also remains dependent on the scene and timestep, whereas APIC and PolyPIC require no blend parameter.

5.8 Three-Dimensional Behaviour

Purpose and setup. The analysis so far has been two-dimensional, where transfer artifacts are easiest to isolate and the sweeps are cheap. This section asks how far those conclusions carry into three dimensions. Rather than repeat the scene catalogue, it uses the dam break alone (Section 5.5), the integrated scene that already combines dissipation, ringing, free-surface motion, and boundary interaction. Re-running the controlled single-effect scenes in three dimensions would mostly reproduce their two-dimensional numbers at far greater cost, whereas the dam break exercises every effect at once and adds what only three dimensions show: free-surface fragmentation into sheets and droplets, and a relatively larger sub-grid null space, since each cell now seeds eight particles rather than four. PolyPIC's three-dimensional port is complete, so all four transfers are compared as in two dimensions.

The configuration mirrors the two-dimensional dam break. The domain is a 64^3 grid at spacing $\Delta x = 1$ with free-slip walls, advanced at the fixed timestep $\Delta t = 0.1$ over 180 time units and seeded at eight particles per cell, for $635\,376$ particles in total. The diagnostics and the late-time window $120 < t \leq 180$ follow Section 5.5.

Expected behaviour. If the per-method orderings are properties of the transfer rules rather than artifacts of two dimensions, they should recur here, with PIC damping the splash most strongly, FLIP retaining the most unresolved motion, and

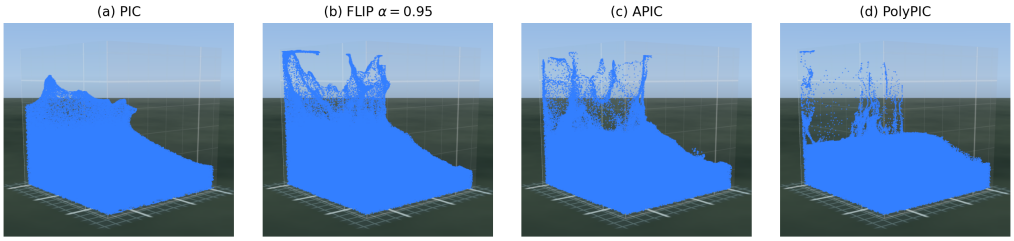


Figure 5.11: Three-dimensional dam break at $t = 25$, near the peak of the splash. (a) PIC collapses into a smooth, coherent mound. (b–d) FLIP, APIC, and PolyPIC break the same splash into sheets and droplets. Free-surface fragmentation is a three-dimensional behaviour the two-dimensional scenes cannot show, and it renders the transfer differences as spray rather than as a number.

APIC and PolyPIC between them. The relative standing of APIC and PolyPIC was found to be scene-dependent in two dimensions (Sections 5.4 and 5.5), set by how much grid-representable cross-mode content the flow excites, so it may shift in three dimensions, where the representable space gains the further cross-modes xz, yz, and xyz that PolyPIC keeps and APIC discards.

Results. The broad ordering recurs. The ringing score is lowest for PIC at 2.0×10^{-7} and highest for FLIP at 1.6×10^{-5}, with the enriched transfers between, and every run stays stable, retaining all $635\,376$ particles with no invalid or out-of-bounds states. The fragmentation that two dimensions cannot show is visible in Figure 5.11: PIC collapses into a smooth, coherent mound, while FLIP, APIC, and PolyPIC break the same splash into sheets and droplets.

Within the enriched transfers the relationship differs from the two-dimensional dam break. There APIC held the lower ringing score and PolyPIC the higher, whereas here the order is reversed, with PolyPIC at 3.1×10^{-6} below APIC at 9.5×10^{-6}. The realised Courant number meanwhile tracks how much motion each transfer preserves: the strongly damped PIC and APIC peak at 6.4 and 6.6, while the less dissipative FLIP and PolyPIC run hotter at 7.8 and 7.3.

Discussion. That the broad ordering recurs in three dimensions shows it is a property of the transfer rules and not of the two-dimensional setting, the cross-dimensional counterpart of the cross-scene consistency the dam break already supports.

The reversal between APIC and PolyPIC is not evidence that one is better. It confirms the finding from the parameter sweeps and the two-dimensional scenes that their relative ringing is scene-dependent and set by the grid-representable cross-mode content the flow excites (Sections 5.4 and 5.7). Three dimensions

enlarge that content with the additional xz, yz, and xyz modes, which PolyPIC retains and APIC drops, shifting the balance towards PolyPIC in this scene. What carries over is the mechanism, not a fixed ranking.

PolyPIC's lower ringing here is not free. At the fixed timestep, preserving more grid-representable motion keeps particle velocities higher, so PolyPIC realises a higher Courant number than APIC and sits closer to the stability limit identified in the parameter study (Section 5.7). Its reduced dissipation therefore trades a margin of stability for the retained motion, an inherent consequence of stepping at a fixed size rather than a separate defect. The three-dimensional dam break thus carries the central two-dimensional findings into the regime the methods are used for in practice: the damping-versus-ringing ordering holds, fragmentation makes the difference visible, and the standing of APIC against PolyPIC remains governed by the represented cross-mode content rather than by a fixed ranking.

5.9 Computational Cost

The quality differences of the preceding sections are paid for in runtime and in memory, and this section reports both. The transfer rule changes the cost of the solver only through two channels, namely the per-particle state it stores and the arithmetic of its two transfer stages. Grid storage and the pressure solve are identical across methods by construction (Section 4.7), so any measured cost difference is attributable to the transfer rather than to the rest of the cycle. The figures below combine the specific implementation, the double-precision particle state, and the diagnostic and export settings of Section 5.1, so they are best read as relative costs within this experimental setup rather than as universal benchmarks.

Runtime is measured as wall-clock time in the deterministic headless solver, with the cycle stages timed separately by a scoped profiler so that the two transfer stages are isolated from the pressure projection. The separation matters because the projection dominates the step. In the two-dimensional dam break it accounts for about sixty per cent of the step and the diagnostic instrumentation for roughly a further fifth, which leaves the two transfer stages together below five per cent of the wall-clock time. Total runtime is therefore a poor measure of transfer cost. The clearest illustration is that PIC has the cheapest transfer of the four methods yet the longest total runtime in that scene, because its strongly damped flow requires more conjugate-gradient iterations in the projection than the motion-preserving methods. The isolated transfer-stage time is the meaningful within-scene quantity, and it is the basis of the comparison reported here. Runtimes are only ever compared between methods of the same scene, since particle count, dimension, and step count differ between scenes.

The per-particle transfer state and the measured transfer-stage cost are collected in Table 5.1. Beyond the single velocity that every method stores, APIC adds the affine velocity matrix and PolyPIC adds the coefficients of its higher cross-modes, counted here as scalars beyond the velocity. The constant PolyPIC mode coincides with the particle velocity, so the multilinear basis contributes three extra modes in two dimensions and seven in three. At eight bytes per double-precision scalar, the transfer state grows from 16 bytes per particle for PIC and FLIP to 64 bytes for PolyPIC in two dimensions, and from 24 to 192 bytes in three. Multiplied by the particle count this gives the instantaneous transfer-state footprint of a run. For the three-dimensional dam break, with $N \approx 6.35 \times 10^5$ particles, the figures above amount to about 15 MB for PIC and FLIP ($635\,376 \times 24$ bytes), 61 MB for APIC ($\times\, 96$ bytes), and 122 MB for PolyPIC ($\times\, 192$ bytes). This is the storage of the transfer representation alone and not the full solver footprint, but it is the part that scales with the particle count and that differs between methods. The analytic counts match the storage layout of the particle record in the implementation.

A live run holds only this single instantaneous state, but replaying or restarting a result without recomputing it requires the particle data to be retained along the trajectory. The number of stored states is fixed by the timestep and the total simulated time as $T/\Delta t$, and each state need carry only the position and velocity of every particle, that is $2d$ scalars in d dimensions, independently of the transfer method, since the affine and higher-mode coefficients are reconstructed from the grid each step. As an analytical baseline, retaining this kinematic state at every step in double precision costs $N \cdot (T/\Delta t) \cdot 2d \cdot 8$ bytes. For the three-dimensional dam break, with $N \approx 6.35 \times 10^5$ particles, $T/\Delta t = 1800$ steps, and $d = 3$, this is about 55 GB per run, on the order of 0.2 TB for the four method runs together. This is an unoptimised upper bound rather than a target, since a practical system stores at a display frame rate rather than at every step, reduces or quantises the precision, and compresses the result. The point is that the dominant storage cost of an experiment is governed by the particle count and the length of the trajectory rather than by the transfer method, whose contribution is the per-particle state of Table 5.1.

The measured transfer cost follows the expected ordering. PIC and FLIP scatter and gather a single velocity and are the cheapest. APIC additionally reconstructs and scatters the affine state and adds a few per cent. PolyPIC additionally solves for and scatters its higher modes and is the most expensive, at roughly 1.7 to 1.8 times the PIC transfer time across the two-dimensional baseline scenes. The PolyPIC solve is a per-mode division over a mass-orthogonal basis rather than a dense inversion [FGG+17], so its overhead stays bounded. The cost grows with dimension, since the multilinear basis carries eight modes in three dimensions against four in two. In the three-dimensional dam break, with about 6.4×10^5 particles, the PolyPIC transfer stages take close to three times the PIC transfer time,

Table 5.1: Per-particle transfer state and measured transfer cost of the four methods. Extra scalars are counted beyond the single velocity that every method stores (two components in two dimensions, three in three). Relative state is the full transfer state, velocity included, divided by that of PIC. The transfer-time ratio is the combined particle-to-grid and grid-to-particle stage time of the two-dimensional dam break relative to PIC, from the scoped profiler. Counts are for the multilinear PolyPIC basis used in the fluid scenes.

	Transfer state	Extra scalars (2D / 3D)	Relative state (2D / 3D)	Transfer time (2D)
PIC	velocity	0 / 0	1 / 1	1.00
FLIP ($\alpha = 0.95$)	velocity	0 / 0	1 / 1	1.01
APIC	+ affine matrix	4 / 9	3 / 4	1.03
PolyPIC	+ cross-modes	6 / 21	4 / 8	1.76

while the total run grows from roughly 4.2 hours for PIC to about 5.0 hours for PolyPIC. At this scale the per-stage differences between PIC, FLIP, and APIC are small enough to sit within the run-to-run variation of multi-hour runs, so only the PolyPIC overhead and the storage growth read as clear effects. The compressed hyperelastic square (Section 5.6) corroborates the runtime trend from its total wall-clock alone, with PolyPIC about 1.7 times PIC using the larger nine-mode quadratic basis of the material-point setting, though that run exposes no per-stage breakdown.

Read together with the quality results, this cost is what APIC and PolyPIC pay for their stability and preserved detail, a modest runtime overhead on a projection-dominated step and a per-particle storage that grows to four times the baseline in two dimensions and eight times in three. Whether that payment is worth making is taken up in the discussion (Section 5.10). Where a cheaper method can be tuned to match the quality, the choice reduces to this cost, and where it cannot, the cost is the price of a result the cheaper method does not reach.

5.10 Discussion

Each scene was built to isolate one criterion and was read on its own terms. Taken together they describe a single trade-off rather than a list of separate outcomes. Across every scene the methods arrange themselves along the axis between dissipation and retained motion that the hypothesis of Section 4.5 anticipated. PIC sits at the dissipative end: it carries by far the largest effective viscosity in the Taylor-Green vortex ($\nu_{\text{eff}} = 1.65$, Section 5.2), loses most of the concentrated rotation in the confined vortex (retaining 0.12 of the core angular momentum against the others' 0.79 to 0.97, Section 5.3), and damps the settling pool and the released

elastic square almost to rest. Its near-zero ringing score is this same dissipation read from the other side, not a sign of fidelity. FLIP sits at the opposite end. It preserves much more motion but carries part of it as grid-unresolved velocity, which surfaces as the highest ringing score in the settling pool and the dam break and as the only raised unresolved fraction in the elastic square.

APIC occupies the position the hypothesis claimed for it, and no single scene could establish this alone. It conserves the concentrated rotation that PIC loses, dissipates almost as little as the least dissipative method ($\nu_{\text{eff}} = 0.022$, against FLIP's 0.102), and still keeps the low ringing of the filtered transfers rather than FLIP's noise. It therefore reaches FLIP-like detail and PIC-like stability at once, with a conservation guarantee neither baseline provides. The controlled scenes were designed to test these properties separately, and the dam break and the parameter study then confirmed that they hold together rather than only one at a time.

The relationship between APIC and PolyPIC is more nuanced, and reading the scenes against one another resolves what any one of them leaves ambiguous. On the smooth conservation measures PolyPIC is consistently a little ahead, with about half the effective viscosity of APIC in the Taylor-Green vortex and two percentage points more core angular momentum in the confined vortex. On ringing the two trade places by scene: APIC has the lower score in the settling pool and the two-dimensional dam break, while PolyPIC has the lower score in the three-dimensional dam break. This is not a contradiction but the central qualification on PolyPIC. Its only addition over APIC is the grid-representable cross-modes the affine state discards, so its advantage appears precisely where the flow excites those modes and is otherwise slight. The three-dimensional dam break, which adds the xz, yz, and xyz modes, is where it shows most, whereas on the smooth two-dimensional fields the two are nearly indistinguishable. PolyPIC's gain over APIC is therefore real but bounded by the interpolation order, and the parameter study confirms that this ordering, unlike the FLIP blend, holds across timestep and resolution rather than only at the baseline.

That robustness is itself a result. The parameter study (Section 5.7) shows the separation between the methods persisting across blend, timestep, and resolution, and the ringing score growing rather than vanishing as the grid is refined, which marks it as a genuine aliasing artifact of the transfer rather than a coarse-grid error. The same study exposes the cost of the one tunable baseline. The FLIP blend is the most parameter-sensitive of the methods, its retained energy collapsing by more than a factor of ten as the timestep is refined, because the small PIC fraction it carries re-anchors the particles to the grid on every step. APIC and PolyPIC reach the favourable region of the detail-versus-noise trade-off with no such parameter to choose. The compressed elastic square (Section 5.6) then shows the ordering surviving outside the incompressible solver entirely, so the behaviour studied here is a property of the particle-grid exchange and not of the fluid setting.

These measurements were taken with the production safeguards deliberately removed (Section 5.1), so that each difference could be attributed to the transfer rule alone. A solver in use would not run so exposed. It would reseed particles to keep the sampling even, and might add a little explicit viscosity or velocity smoothing to suppress noise. Most such remedies stabilise a noisy transfer by re-adding dissipation, which discards the very detail the transfer was chosen to keep. The practical question is therefore not whether FLIP can be stabilised, which is straightforward, but whether it can be stabilised without giving back the detail that separates it from PIC. APIC closes the noise gap and the detail gap at once and without a stabiliser to tune, which is the practical form of its advantage.

The stability behaviour traced to the Courant number in the earlier sections also has a direct practical remedy. The parameter study placed the onset of instability where the late-time Courant number passes one, and the three-dimensional dam break ran at peak Courant numbers between six and eight, highest for the least dissipative transfers because preserving more motion keeps particle speeds higher at a fixed step. An adaptive timestep with substepping holds the realised Courant number below this limit and removes the instability without adding dissipation, but it does so by taking more steps per frame, so the stability is bought with run time. The transfer-stage overhead measured in Section 5.9 is therefore not the whole cost of running a less dissipative method near its limit.

For deformable materials the choice between APIC and PolyPIC carries a consideration beyond cost. As the elastic square shows, PolyPIC does not merely transfer the same material more faithfully. Its additional modes change the material response, so it is not a drop-in replacement that leaves authored behaviour unchanged. Because the modes are backward compatible, with APIC recovered exactly by dropping the cross-modes, the affine behaviour can still be reproduced unchanged, while a material authored to exploit the extra modes could expose deformation the affine transfer cannot represent, giving an artist more control rather than less. Its naturally lower dissipation has the same double character. Left unchecked it yields scenes that retain more energy than a physical material would and that must then be damped, but that same retention sits closer to the lossless ideal, so the property that needs taming is also what makes the simulation more accurate and lets damping be added deliberately rather than inherited from the transfer.

Whether such properties justify adopting a method in production depends on factors beyond its numerical quality. An engineer at the studio behind the *Moana* films, asked why the sequel did not adopt a higher-order transfer, gave a high-level account that places method quality inside a larger decision (Y. K. Li, personal communication, May 20, 2026). The first film used a custom in-house solver built on APIC for ocean and water effects at a scale the commercial tools of the time could not handle. The sequel instead used a commercial package, because

it had since gained native APIC support and integrated more smoothly with the surrounding effects pipeline, so that a second custom solver was unnecessary for the scale the production required. A higher-order transfer such as PolyPIC would be taken up only if a future production needed the additional detail and vortices it provides and no adequate tool already offered them. The deciding questions were what the production required, whether the technique could be made art-directable, whether an off-the-shelf option was good enough, and where it ranked against everything else competing for finite development effort. The transfer-quality differences measured here inform the first of those questions but do not settle the rest.

The reach of these findings is bounded by the setup that produced them. The analysis is primarily two-dimensional, where the artifacts are cleanest and the sweeps are dense, with three dimensions sampled in a single integrated scene. The solver is inviscid and free-slip, so viscous and wall-driven flows are out of scope. Every run shares one machine and build. Within these bounds the chapter has established where each transfer sits on the dissipation-detail axis and how robustly it stays there, which the conclusion carries forward.

Chapter 6

Conclusion

This chapter summarizes the findings of the transfer-method comparison, discusses the limitations of the evaluation, and outlines directions for future work.

6.1 Summary

This thesis evaluated how particle-grid transfer affects hybrid simulation by comparing PIC, FLIP, APIC, and PolyPIC in controlled two- and three-dimensional fluid experiments and a deformable-material test. With the remaining solver configuration fixed, the comparison measured numerical dissipation, angular-momentum retention, ringing, stability, and computational and storage cost.

The results expose a trade-off between damping and retained motion. PIC suppressed ringing through strong dissipation, while FLIP preserved more motion than PIC but produced the largest grid-unresolved component and was sensitive to its blend and the timestep. APIC preserved rotation and detail with substantially less ringing. PolyPIC further reduced dissipation and sometimes retained more detail, but its ringing relative to APIC depended on the scene, and its additional modes increased storage and transfer cost. The comparison therefore supports APIC as a robust compromise rather than a universally superior method, while PolyPIC is advantageous when its additional representable modes justify their cost.

6.2 Limitations

The conclusions are bounded by the experimental design. Most measurements use a finite set of two-dimensional fluid scenes with an inviscid solver and free-slip boundaries. Three-dimensional behaviour is represented by one dam-break experiment, and the deformable-material comparison by one material point method benchmark. The parameter study varies one factor at a time in the settling-pool

scene. The results therefore characterise the tested transfer implementations and regimes, but do not establish a scene-independent ranking.

EPIC is an experimental evaluation platform rather than a general-purpose animation engine. Its test scenes are hard-coded for controlled comparison rather than general scene construction, and the experiments exclude the safeguards of a production solver. This improves reproducibility and the attribution of observed differences to the transfer rule, but limits conclusions about broader geometries, materials, and production workflows. The runtime measurements are also specific to the implementation, machine, and build, so only their relative ordering within this setup is meaningful.

6.3 Future Work

The deformable-material evaluation could be extended to three dimensions. Additional three-dimensional material point method scenes would test whether the differences between APIC and PolyPIC observed in the compressed square persist with a larger representable mode space and more complex deformation.

EPIC could also support a broader range of controlled experiments by exposing scene-design controls through its interactive user interface. Configurable geometry, initial conditions, material parameters, and solver settings would reduce the dependence on hard-coded scenes while preserving the deterministic configurations and headless execution used for quantitative comparison.

More recent extensions such as IPIC, discussed in Section 2.3, demonstrate that particle-grid methods continue to evolve beyond PolyPIC. Future versions of EPIC could therefore incorporate further transfer schemes and evaluate them under the same controlled protocol.

Bibliography

[BCDG13] A Barreiro, AJC Crespo, JM Domínguez, and M Gómez-Gesteira. "Smoothed particle hydrodynamics for coastal engineering problems". In: *Computers & Structures* 120 (2013), pp. 96–106 (cited on page 6).

[BK16] Jan Bender and Dan Koschier. "Divergence-free SPH for incompressible and viscous fluids". In: *IEEE Transactions on Visualization and Computer Graphics* 23.3 (2016), pp. 1193–1206 (cited on page 6).

[Bra88] J.U Brackbill. "The ringing instability in particle-in-cell calculations of low-speed flow". In: *Journal of Computational Physics* 75.2 (1988), pp. 469–492. ISSN: 0021-9991. DOI: https://doi.org/10.1016/0021-9991(88)90123-4. URL: https://www.sciencedirect.com/science/article/pii/0021999188901234 (cited on pages 45, 59).

[BR86] Jeremiah U Brackbill and Hans M Ruppel. "FLIP: A method for adaptively zoned, particle-in-cell calculations of fluid flows in two dimensions". In: *Journal of Computational physics* 65.2 (1986), pp. 314–343 (cited on pages 2, 3, 5, 8, 31).

[Bri15] Robert Bridson. *Fluid simulation for computer graphics*. AK Peters/CRC Press, 2015 (cited on pages 1, 5, 7, 11–16, 20–24, 27, 30, 32, 33, 52, 71).

[CML+17] Andrés Caballero, Wenbin Mao, Liang Liang, John Oshinski, Charles Primiano, Raymond McKay, Susheel Kodali, and Wei Sun. "Modeling left ventricular blood flow using smoothed particle hydrodynamics". In: *Cardiovascular engineering and technology* 8.4 (2017), pp. 465–479 (cited on page 6).

[FSN17] Ben Frost, Alexey Stomakhin, and Hiroaki Narita. "Moana: performing water". In: *ACM SIGGRAPH 2017 Talks*. 2017, pp. 1–2 (cited on page 2).

[FGG+17] Chuyuan Fu, Qi Guo, Theodore Gast, Chenfanfu Jiang, and Joseph Teran. “A polynomial particle-in-cell method”. In: *ACM Transactions on Graphics (TOG)* 36.6 (2017), pp. 1–12 (cited on pages 2, 3, 5, 8, 12, 35–39, 51, 54, 58, 66, 69, 75).

[HW+65] Francis H Harlow, J Eddie Welch, et al. “Numerical calculation of time-dependent viscous incompressible flow of fluid with free surface”. In: *Physics of fluids* 8.12 (1965), p. 2182 (cited on pages 5, 7, 13–15).

[Har62] Francis H. Harlow. *The particle-in-cell method for numerical solution of problems in fluid dynamics*. Tech. rep. Los Alamos Scientific Laboratory, 1962 (cited on pages 2, 3, 5, 8, 29).

[JSS+15] Chenfanfu Jiang, Craig Schroeder, Andrew Selle, Joseph Teran, and Alexey Stomakhin. “The affine particle-in-cell method”. In: *ACM Transactions on Graphics (TOG)* 34.4 (2015), pp. 1–10 (cited on pages 2, 3, 5, 8, 12, 13, 31, 33–35, 43, 44, 57, 58, 62).

[JST17] Chenfanfu Jiang, Craig Schroeder, and Joseph Teran. “An angular momentum conserving affine-particle-in-cell method”. In: *Journal of Computational Physics* 338 (2017), pp. 137–164 (cited on pages 32, 34, 35, 37, 43–45, 58).

[KDK+19] Christian Kühnlein, Willem Deconinck, Rupert Klein, Sylvie Malardel, Zbigniew P Piotrowski, Piotr K Smolarkiewicz, Joanna Szmelter, and Nils P Wedi. “FVM 1.0: a nonhydrostatic finite-volume dynamical core for the IFS”. In: *Geoscientific Model Development* 12.2 (2019), pp. 651–676 (cited on pages 1, 7).

[Li24] Yining Karl Li. *Moana 2*. Updated August 10, 2025 with SIGGRAPH 2025 content. Dec. 18, 2024. URL: `https://blog.yiningkarlli.com/2024/12/moana-2.html` (visited on 05/11/2026) (cited on page 2).

[Mon05] Joe J Monaghan. “Smoothed particle hydrodynamics”. In: *Reports on progress in physics* 68.8 (2005), pp. 1703–1759 (cited on page 6).

[MCG03] Matthias Müller, David Charypar, and Markus Gross. “Particle-based fluid simulation for interactive applications”. In: *Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation*. 2003, pp. 154–159 (cited on page 6).

[MST04] Matthias Müller, Simon Schirm, and Matthias Teschner. “Interactive blood simulation for virtual surgery based on smoothed particle hydrodynamics”. In: *Technology and Health Care* 12.1 (2004), pp. 25–31 (cited on page 6).

[STBA24] Sergio Sancho, Jingwei Tang, Christopher Batty, and Vinicius C Azevedo. “The Impulse Particle-In-Cell Method”. In: *Computer Graphics Forum*. Vol. 43. 2. Wiley Online Library. 2024, e15022 (cited on page 9).

[SSC+13] Alexey Stomakhin, Craig Schroeder, Lawrence Chai, Joseph Teran, and Andrew Selle. “A material point method for snow simulation”. In: *ACM Transactions on Graphics (TOG)* 32.4 (2013), pp. 1–10 (cited on pages 2, 5, 9, 13, 39, 40).

[SCS94] Deborah Sulsky, Zhen Chen, and Howard L Schreyer. “A particle method for history-dependent materials”. In: *Computer methods in applied mechanics and engineering* 118.1-2 (1994), pp. 179–196 (cited on pages 2, 5, 9, 13).

[TG37] Geoffrey Ingram Taylor and Albert Edward Green. “Mechanism of the production of small eddies from large ones”. In: *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 158.895 (Feb. 1937), pp. 499–521. ISSN: 0080-4630. DOI: 10.1098/rspa.1937.0036. eprint: https://royalsocietypublishing.org/rspa/article-pdf/158/895/499/34566/rspa.1937.0036.pdf. URL: https://doi.org/10.1098/rspa.1937.0036 (cited on page 53).

[Wen08] John F Wendt. *Computational fluid dynamics: an introduction*. Springer Science & Business Media, 2008 (cited on pages 1, 11, 27).

[ZB05] Yongning Zhu and Robert Bridson. “Animating sand as a fluid”. In: *ACM Transactions on Graphics (TOG)* 24.3 (2005), pp. 965–972 (cited on pages 2, 5, 8, 11–13, 26, 31, 33, 52, 71).

AI Disclaimer Generative AI tools were used as auxiliary tools for structural and linguistic revision, feedback, and software debugging. They were not used to generate experimental data or determine scientific results or conclusions.